

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**CyberCAN Defense: Phát hiện và phòng
thủ xâm nhập mạng nội bộ ô tô**

Nhóm 12

Quách Ngọc Quang	22022132
Nguyễn Tuấn Phong	22022160
Nguyễn Minh Phúc	22022178
Đoàn Đức Mạnh	22022167

Cơ sở đo lường và điều khiển số

Lớp: ELT3207_1

Giảng viên: TS. Phạm Duy Hưng

Mentor: Vũ Minh Ngọc

Hà Nội, 2025

LỜI CAM ĐOAN

Chúng tôi cam đoan dự án "CyberCAN Defense: Phát hiện và Phòng thủ Xâm nhập mạng nội bộ ô tô" là công trình nghiên cứu của chính nhóm chúng tôi thực hiện, dưới sự dẫn dắt của anh Vũ Minh Ngọc - Mentor FPT Software và thầy Phạm Duy Hưng - giảng viên bộ môn: Cơ sở đo lường và điều khiển số. Các số liệu, nội dung và kết quả trình bày trong dự án là trung thực, chính xác và không vi phạm quy định về đạo đức nghiên cứu khoa học. Chúng tôi xin chịu trách nhiệm hoàn toàn với lời cam đoan này.

LỜI CẢM ƠN

Để hoàn thành tốt bài tập lớn môn Cơ sở đo lường và điều khiển số – một học phần có vai trò đặc biệt quan trọng trong chương trình đào tạo ngành Kỹ thuật máy tính, nhóm chúng em xin gửi lời cảm ơn sâu sắc đến thầy Phạm Duy Hưng, giảng viên phụ trách môn học, người đã luôn theo sát, hướng dẫn tận tình và hỗ trợ nhóm em trong suốt môn học. Thầy không chỉ truyền đạt những kiến thức lý thuyết cốt lõi một cách dễ hiểu, mà còn đưa ra nhiều gợi ý thiết thực, giúp chúng em kết nối giữa lý thuyết và thực tiễn, định hướng cách tư duy và tiếp cận vấn đề một cách logic và hiệu quả. Sự nghiêm túc, tâm huyết và tinh thần trách nhiệm của thầy chính là động lực to lớn để nhóm em nỗ lực hoàn thành môn học với tinh thần tốt nhất.

Nhóm em cũng xin gửi lời cảm ơn chân thành đến anh Vũ Minh Ngọc, mentor tại công ty FPT Software, người đã nhiệt tình đồng hành và hỗ trợ nhóm em trong việc áp dụng kiến thức học thuật vào môi trường thực tế của doanh nghiệp. Anh đã dành thời gian quý báu để góp ý, chia sẻ kinh nghiệm và tạo điều kiện thuận lợi để nhóm em có thể tiếp cận với các công cụ, thiết bị và mô hình đo lường thực tế, từ đó giúp nhóm hoàn thiện bài tập với tính ứng dụng cao và sát với thực tiễn kỹ thuật hiện nay. Sự hỗ trợ của anh không chỉ giúp chúng em hoàn thành bài tập lớn cuối kỳ, mà còn mở ra cái nhìn thực tế và định hướng nghề nghiệp trong tương lai.

Dù đã rất nỗ lực và đầu tư nghiêm túc trong quá trình thực hiện, bài tập lớn cuối kỳ của nhóm em chắc chắn vẫn không tránh khỏi những hạn chế và thiếu sót nhất định. Nhóm chúng em rất mong nhận được những nhận xét, góp ý chân thành từ thầy Hưng và anh Ngọc để nhóm em rút kinh nghiệm cho những bài tập lớn khác và nghề nghiệp sau này.

Một lần nữa, nhóm em xin bày tỏ lòng biết ơn sâu sắc đến tất cả những người đã hỗ trợ, tạo điều kiện và đồng hành cùng chúng em trong quá trình thực hiện bài tập lớn cuối kỳ này.

MỤC LỤC

Chương 1: Giới thiệu chung	1
I. Bối cảnh	1
II. Giải pháp	1
Chương 2: Giao thức CAN	3
I. Khái niệm	3
II. Nguyên tắc hoạt động	4
III. Ứng dụng trong Automotive: Hệ thống ô tô	7
Chương 3: Bảo mật trong hệ thống mạng CAN	10
I. Các vấn đề về bảo mật trong hệ thống mạng CAN	10
II. Các phương pháp bảo mật trong hệ thống mạng CAN	10
1. Mã hóa thông tin (Encryption)	10
2. Xác thực thông điệp (Message Authentication)	11
3. Sử dụng bộ đếm tin nhắn đơn giản (Simple Message Counter)	12
4. Phát hiện xâm nhập (Intrusion Detection System - IDS)	13
5. Phân vùng mạng (Segmentation)	13
6. Chữ ký số (Digital Signatures)	14
7. Giao thức bảo mật lớp cao (Higher-layer Security Protocols)	14
Chương 4: CAN trong STM32F103	16
I. Giới thiệu	16
II. Các thanh ghi	17
1. Điều khiển & trạng thái tổng	17
2. Định thời Bit	17
3. Gửi (TX)	18
4. Nhận (RX)	19
5. Bộ lọc	20
6. Ngắt	22
Chương 5: Thiết kế hệ thống	23
I. Hệ thống phần cứng	23
1. Linh kiện sử dụng	23
2. Sơ đồ mạch	25
II. Hệ thống phần mềm	28
1. Giao diện truyền nhận CAN chưa có bảo vệ	28
2. Giao diện truyền nhận CAN có bảo vệ (Protected CAN)	29
3. Giao diện tấn công mạng CAN (CAN Attack Web Interface)	30
4. Cơ sở dữ liệu	31
III. Hệ thống đo đạc	33
1. Logic Analyzer 8 Kênh	33
2. Logic 2	35
IV. Nguyên lý hoạt động của hệ thống	36
1. Giai đoạn giao tiếp CAN cơ bản (Chưa có bảo vệ)	36

2.	Giai đoạn tấn công mạng CAN	36
3.	Giai đoạn bảo vệ (CAN có phòng thủ)	37
4.	Tổng kết nguyên lý	38
V.	Mã nguồn phát triển của hệ thống	39
Chương 6: Thực thi, đo lường và đánh giá		40
I.	Giao tiếp 2 chiều	40
II.	Tấn công	42
III.	Phòng thủ	44
Chương 7: Tổng kết và hướng phát triển		46
I.	Tổng kết	46
II.	Hướng phát triển trong tương lai	47
Tài liệu tham khảo		48

DANH SÁCH HÌNH ẢNH

1	Mạng CAN	3
2	Standard CAN: 11-Bit Identifier	5
3	Extended CAN: 29-Bit Identifier	5
4	CAN FD (Flexible Data CAN)	6
5	Mô hình mạng CAN đơn giản trong hệ thống ô tô	9
6	Sơ đồ tổng quát hệ thống phần cứng.	23
7	STM32F103	24
8	MCP2551	25
9	Can 2 chiều	26
10	Mạch thực tế truyền nhận CAN.	26
11	Sơ đồ mạch truyền Can có thêm hacker	27
12	Mạch thực tế truyền Can có thêm hacker	27
13	Giao diện truyền nhận CAN chưa có bảo vệ	28
14	Giao diện truyền nhận CAN có bảo vệ	30
15	Giao diện mô phỏng tấn công mạng CAN	30
16	Logic analyzer	32
17	Logic analyzer	34
18	Logic analyzer	35
19	Mạch thực hành giao tiếp giữa 2 vi điều khiển.	40
20	Giao tiếp giữa 2 vi điều khiển.	41
21	Tín hiệu nhận được trên Web.	42
22	Mạch giao tiếp CAN thêm phần tử Hacker.	43
23	Giao diện tấn công với cyclic 500ms.	43
24	Tín hiệu tấn công trên CAN bus với chu kỳ 500ms.	44
25	Giao diện với cơ chế phòng thủ.	45

DANH SÁCH LIÊN KẾT

- [Github](#)
- [Video truyền 2 chiều trong CAN](#)
- [Video truyền CAN và bị Replay Attack](#)
- [Video phòng thủ](#)

Chương 1: Giới thiệu chung

I. Bối cảnh

Trong kỷ nguyên của xe hơi hiện đại, các hệ thống điều khiển điện tử (ECU – Electronic Control Unit) đóng vai trò trung tâm trong việc điều phối các chức năng từ đơn giản như điều hòa không khí, đèn pha cho đến các hệ thống phức tạp như hỗ trợ lái tự động (ADAS), chống bó cứng phanh (ABS), kiểm soát lực kéo (TCS), và túi khí (Airbag).

Để các ECU này có thể giao tiếp hiệu quả, giao thức Controller Area Network (CAN) được sử dụng như một xương sống truyền thông nội bộ trong xe hơi. CAN cung cấp cơ chế truyền thông đa điểm tốc độ cao, đáng tin cậy, và thời gian thực giữa các bộ điều khiển. Tuy nhiên, khi các phương tiện ngày càng trở nên kết nối nhiều hơn – với các cổng OBD-II, kết nối Bluetooth, Wi-Fi, 4G/5G, và thậm chí cập nhật OTA – nguy cơ bị tấn công mạng ngày càng cao.

Mặc dù CAN được thiết kế tối ưu cho tính thời gian thực và độ tin cậy, nhưng nó không có cơ chế bảo mật tích hợp, như:

- Không có xác thực (Authentication) giữa các node.
- Không có mã hóa (Encryption) dữ liệu.
- Không có phân quyền truy cập (Access control).

Do đó, kẻ tấn công chỉ cần truy cập vật lý hoặc gián tiếp vào mạng CAN là có thể thực hiện các hành vi nguy hiểm như:

- Giả mạo gói tin (Spoofing) để điều khiển ECU trái phép.
- Tấn công từ chối dịch vụ (DoS) làm treo hoặc quá tải hệ thống.
- Nghe lén dữ liệu (Eavesdropping) để phân tích hành vi xe.

Trước bối cảnh đó, yêu cầu cấp thiết đặt ra là cần phát triển các giải pháp bảo mật mạng CAN. Một số giải pháp hiện nay trên thế giới như: Hệ thống phát hiện xâm nhập (IDS), mã hóa giao tiếp trên CAN hay tự động phân tích hành vi bất thường trong quá trình vận hành đã được áp dụng và triển khai.

II. Giải pháp

Trước các mối đe dọa an ninh mạng ngày càng gia tăng đối với hệ thống xe hơi, đặc biệt là với giao thức truyền thông CAN bus, nhiều giải pháp bảo mật đã được nghiên

cứu và triển khai trên toàn thế giới. Các giải pháp này tập trung vào việc phát hiện, ngăn chặn, và giảm thiểu tác động của các cuộc tấn công mạng vào hệ thống xe hơi. Dưới đây là các nhóm giải pháp chính:

- a. **Hệ thống phát hiện xâm nhập (IDS – Intrusion Detection System):** IDS là một trong những hướng tiếp cận phổ biến nhất trong bảo mật mạng CAN. Hệ thống này giám sát dữ liệu truyền trên CAN bus và tìm kiếm các dấu hiệu bất thường dựa trên: phân tích thống kê tần suất gói tin, phân tích hành vi (behavioral analysis) hay học máy (machine learning) để phân loại hoạt động hợp lệ và độc hại.
- b. **Mã hóa dữ liệu truyền trên CAN bus:** Một số nghiên cứu đề xuất áp dụng kỹ thuật mã hóa nhẹ (lightweight encryption) để bảo vệ nội dung gói tin truyền qua mạng CAN. Tuy nhiên, giải pháp này gặp nhiều thách thức do: giới hạn băng thông của CAN (chỉ 8 byte dữ liệu/gói) và yêu cầu thời gian thực trong hệ thống xe.
- c. **Xác thực thông điệp (Message Authentication):** Thay vì chỉ mã hóa, một số giải pháp tập trung vào việc gắn chữ ký số nhẹ (Message Authentication Codes – MAC) vào các gói tin CAN, nhằm đảm bảo rằng dữ liệu đến từ nguồn hợp lệ và chưa bị chỉnh sửa. Điều này giúp chống lại các cuộc tấn công spoofing và replay.
- d. **Cách ly và phân vùng mạng (Segmentation & Isolation):** Trong các hệ thống xe hơi hiện đại, việc phân chia mạng nội bộ thành các miền (domains), như miền điều khiển thân xe, động cơ, thông tin giải trí... giúp hạn chế phạm vi lây lan của tấn công, giảm thiểu rủi ro.
- e. **Giám sát hành vi dựa trên định thời (Timing-based anomaly detection):** Nhiều tấn công làm thay đổi chu kỳ truyền hoặc độ trễ của gói tin. Việc theo dõi thời gian truyền và kiểm tra sự bất thường trong mẫu định thời cũng là một kỹ thuật hữu ích trong việc phát hiện xâm nhập.
- f. **Giải pháp từ nhà sản xuất (OEM-level Security):** Một số hãng xe và nhà cung cấp ECU đã tích hợp các công nghệ bảo mật trực tiếp trong phần cứng như: bộ mã hóa CAN tích hợp, kiến trúc phần mềm có xác thực hai chiều giữa các ECU và cập nhật phần mềm an toàn (Secure OTA).

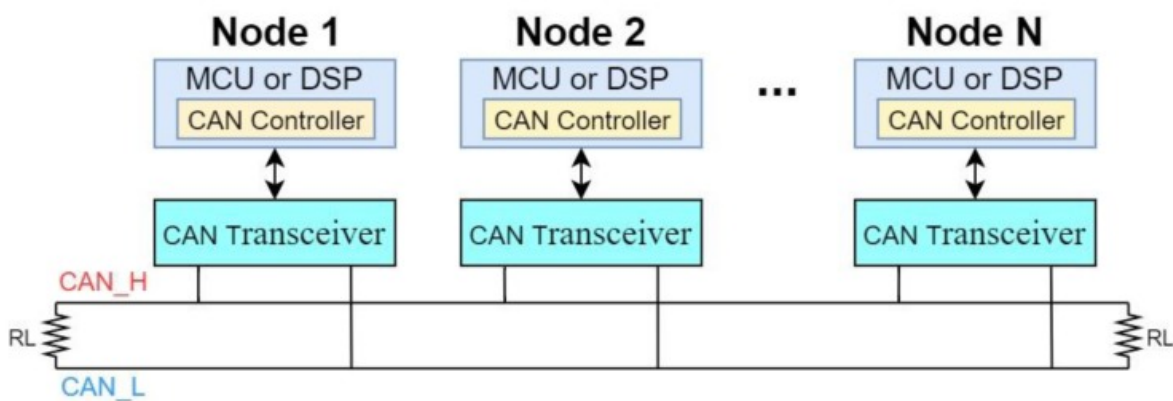
Mặc dù nhiều giải pháp bảo mật mạng CAN đã được nghiên cứu và triển khai, không có giải pháp nào hoàn hảo. Việc kết hợp nhiều lớp bảo mật – từ mã hóa, xác thực, đến phát hiện xâm nhập – cùng với thiết kế hệ thống mạng thông minh và có phân vùng là hướng đi tiềm năng để nâng cao an ninh cho xe hơi hiện đại.

Chương 2: Giao thức CAN

I. Khái niệm

CAN (Controller Area Network) là một giao thức truyền thông nối tiếp được phát triển bởi hãng **Bosch** vào những năm 1980 để giải quyết nhu cầu trao đổi dữ liệu giữa các thiết bị điện tử trong xe ô tô. Trước khi có CAN, các hệ thống điều khiển trong xe phải kết nối trực tiếp với nhau bằng dây dẫn, gây tốn kém và khó bảo trì. Việc sử dụng CAN cho phép các bộ điều khiển (ECU) chia sẻ dữ liệu với nhau thông qua một đường truyền chung, từ đó giảm thiểu số lượng dây, tăng độ tin cậy và khả năng mở rộng hệ thống.

Giao thức CAN hiện được chuẩn hóa trong tiêu chuẩn **ISO 11898** và đã vượt ra khỏi ngành công nghiệp ô tô, được ứng dụng rộng rãi trong công nghiệp tự động hoá, robot, y tế, hàng không và các hệ thống nhúng.



Hình 1: Mạng CAN

Đặc điểm nổi bật của giao thức CAN:

- **Hoạt động theo mô hình đa master (multi-master):** Mọi thiết bị (node) trên mạng đều có quyền khởi tạo truyền dữ liệu. Điều này giúp mạng CAN linh hoạt hơn so với các mô hình truyền thống chỉ có một master điều khiển.
- **Truyền thông không định địa chỉ thiết bị:** Khác với giao thức UART hoặc I2C, CAN không dùng địa chỉ thiết bị. Thay vào đó, mỗi khung dữ liệu mang một **ID (identifier)** đại diện cho nội dung thông điệp. Các node sẽ lọc và xử lý dữ liệu theo ID mà chúng quan tâm. Điều này giúp tăng tính linh hoạt và khả năng mở rộng hệ thống.
- **Cơ chế ưu tiên truyền:** Trong trường hợp nhiều node cùng muốn truyền dữ liệu, giao thức CAN sẽ so sánh ID theo bit từng bước để quyết định thiết bị nào được phép tiếp tục truyền. ID có giá trị nhị phân thấp hơn (ưu tiên cao hơn) sẽ

giành quyền truy cập bus. Quá trình này gọi là **arbitration** và diễn ra hoàn toàn tự động mà không gây xung đột.

- **Phát hiện và xử lý lỗi mạnh mẽ:** CAN có nhiều cơ chế phát hiện lỗi như:
 - **Bit monitoring:** Kiểm tra sự khác biệt giữa bit gửi và nhận.
 - **CRC check:** Kiểm tra tính toàn vẹn khung dữ liệu.
 - **ACK check:** Đảm bảo dữ liệu được ít nhất một node khác nhận thành công.
 - **Form check:** Kiểm tra định dạng của khung dữ liệu.

Nếu phát hiện lỗi, node sẽ tự động phát một **Error Frame** và yêu cầu gửi lại.

- **Tốc độ truyền dữ liệu linh hoạt:** Giao thức CAN hỗ trợ nhiều tốc độ khác nhau tùy theo chiều dài đường truyền. Trên thực tế:
 - Với khoảng cách ngắn (dưới 40m), có thể đạt tốc độ tối đa **1 Mbps**.
 - Với khoảng cách dài (lên đến 1 km), tốc độ có thể giảm xuống khoảng **50 kbps**.

Điều này làm cho CAN phù hợp với cả hệ thống nhỏ lẫn hệ thống phân tán rộng.

- **Giao tiếp đồng bộ và thời gian thực:** CAN sử dụng chuẩn mã hóa NRZ (Non-Return to Zero) và kỹ thuật **bit stuffing** để đồng bộ hóa giữa các node mà không cần xung clock ngoài. Điều này giúp đảm bảo giao tiếp thời gian thực, đặc biệt quan trọng trong các hệ thống an toàn như phanh ABS, điều khiển động cơ hay túi khí.
- **Chi phí thấp, độ tin cậy cao:** Nhờ vào khả năng xử lý lỗi, giảm thiểu dây nối và chi phí phần cứng, CAN trở thành lựa chọn hàng đầu cho các hệ thống nhúng cần độ tin cậy cao với chi phí hợp lý.

II. Nguyên tắc hoạt động

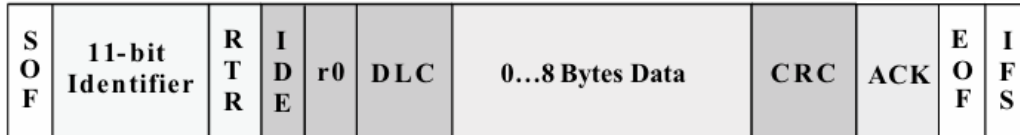
1. Cấu trúc khung dữ liệu (Data Frame)

Trường	Vai trò
Start of Frame	Xác định bắt đầu một khung dữ liệu.
Arbitration Field	Chứa ID và bit RTR (Remote Transmission Request).
Control Field	Xác định độ dài dữ liệu.
Data Field	Dữ liệu truyền đi (từ 0 đến 8 byte).
CRC Field	Dùng để kiểm tra lỗi dữ liệu truyền.
ACK Field	Thiết bị nhận phản hồi xác nhận khung hợp lệ.
End of Frame	Đánh dấu kết thúc khung dữ liệu.

2. Phân loại ID

Trong giao thức CAN, mỗi khung dữ liệu mang một mã định danh (ID - *Identifier*) thay vì địa chỉ thiết bị. ID được sử dụng để xác định nội dung thông điệp, phân mức ưu tiên trong quá trình tranh chấp truyền (*arbitration*) và giúp các node lọc dữ liệu cần thiết.

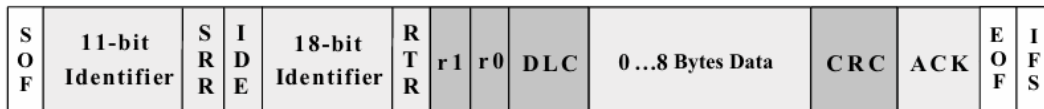
- **Standard ID (11 bit):**



Hình 2: Standard CAN: 11-Bit Identifier

- Là định dạng gốc của chuẩn CAN theo ISO 11898-1, còn gọi là **Base Frame Format**.
- Gồm 11 bit, cho phép tối đa $2^{11} = 2048$ mã định danh khác nhau.
- Được sử dụng rộng rãi trong các hệ thống nhỏ, số lượng thiết bị hoặc thông điệp ít.
- Khung dữ liệu ngắn giúp giảm thời gian truyền, tăng hiệu quả mạng.

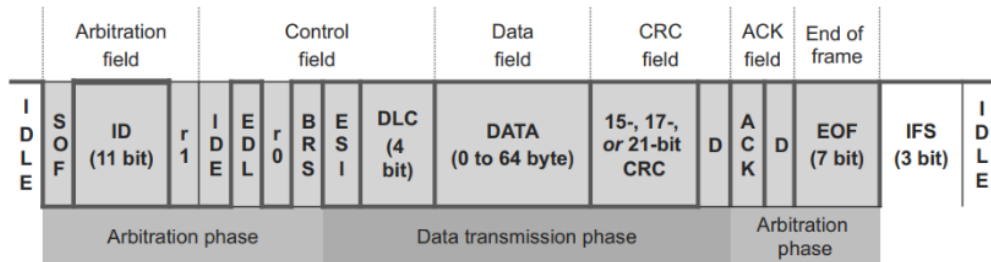
- **Extended ID (29 bit):**



Hình 3: Extended CAN: 29-Bit Identifier

- Mở rộng từ chuẩn CAN cơ bản để đáp ứng nhu cầu phức tạp hơn.
- Gồm 11 bit gốc + 18 bit mở rộng $\Rightarrow$ tổng 29 bit.
- Cho phép tối đa $2^{29} \approx 537$ triệu mã định danh khác nhau.
- Thường dùng trong các hệ thống lớn như xe hiện đại, hệ thống công nghiệp quy mô lớn.
- Phân biệt bằng bit điều khiển IDE (Identifier Extension): nếu $IDE = 1 \Rightarrow$ sử dụng Extended ID.

3. CAN FD (Flexible Data-rate)



Hình 4: CAN FD (Flexible Data CAN)

CAN FD là phiên bản mở rộng của chuẩn CAN cổ điển, do Bosch phát triển vào năm 2012 và được tiêu chuẩn hóa trong ISO 11898-1:2015. Nó khắc phục các hạn chế về tốc độ và dung lượng dữ liệu.

Những điểm cải tiến chính:

- **Tăng dung lượng dữ liệu:**

- CAN cổ điển giới hạn ở **8 byte** dữ liệu.
- CAN FD hỗ trợ tối đa **64 byte** dữ liệu mỗi khung.

- **Tốc độ truyền linh hoạt:**

- Giai đoạn *arbitration* giữ tốc độ thấp để đảm bảo tương thích.
- Giai đoạn *data phase* có thể đạt tốc độ cao hơn (**tối đa 8 Mbps** hoặc hơn).

- **Tính tương thích ngược:**

- Có thể hoạt động chung với thiết bị chỉ hỗ trợ CAN cổ điển.
- Bit FDF (FD Format) giúp thiết bị cổ điển nhận biết và bỏ qua khung FD.

- **Cấu trúc khung mới:**

- Bổ sung trường FDF, BRS (Bit Rate Switch) để xác định khung FD và tốc độ cao.
- CRC tăng lên 17-bit hoặc 21-bit tùy chiều dài dữ liệu.

Tổng kết:

- **Standard ID (11 bit):** Dành cho hệ thống đơn giản, số lượng thiết bị ít, truyền ổn định.

- **Extended ID (29 bit):** Thích hợp cho hệ thống phức tạp, yêu cầu nhiều loại thông điệp.
- **CAN FD:** Lý tưởng cho ứng dụng thời gian thực, yêu cầu truyền dữ liệu lớn, tốc độ cao.

4. Truyền và nhận dữ liệu

- Tất cả thiết bị đều giám sát liên tục bus CAN.
- Thiết bị bắt đầu truyền khi phát hiện bus rảnh.
- Cơ chế **arbitration** dựa vào ID: thiết bị có ID nhỏ hơn sẽ được ưu tiên.
- Sử dụng mã hóa NRZ và kỹ thuật **bit stuffing** để đảm bảo đồng bộ.

5. Phát hiện lỗi

- **Bit Monitoring:** So sánh bit truyền và bit nhận trên bus.
- **CRC Checking:** Kiểm tra tính toàn vẹn của dữ liệu.
- **Form Checking:** Đảm bảo khung dữ liệu đúng định dạng.
- **ACK Checking:** Thiết bị nhận phải xác nhận bằng cách gửi ACK.

6. Giao tiếp đa điểm

Giao tiếp theo mô hình **bus**, nơi nhiều node kết nối chung trên hai dây tín hiệu CAN (CAN_H và CAN_L). Hai đầu của bus cần gắn điện trở **120Ω** để triệt tiêu phản xạ và đảm bảo tín hiệu ổn định.

III. Ứng dụng trong Automotive: Hệ thống ô tô

Trong các phương tiện giao thông hiện đại, đặc biệt là ô tô, giao thức **CAN (Controller Area Network)** đóng vai trò là xương sống trong mạng truyền thông nội bộ. Với khả năng truyền thông tin hiệu quả, đáng tin cậy và chi phí thấp, CAN đã trở thành tiêu chuẩn công nghiệp trong các hệ thống điện tử trên xe.

Các hệ thống tiêu biểu sử dụng giao thức CAN

- **ECU động cơ (Engine Control Unit):** Điều khiển hoạt động của động cơ như phun nhiên liệu, đánh lửa, kiểm soát khí xả, v.v.
- **Hệ thống túi khí (Airbag System):** Giao tiếp giữa các cảm biến va chạm, bộ xử lý và bộ kích hoạt túi khí để đảm bảo phản ứng tức thời khi xảy ra va chạm.
- **Hệ thống phanh ABS/ESP:** Trao đổi dữ liệu liên tục giữa cảm biến tốc độ bánh xe, ECU phanh và các mô-đun ổn định thân xe để ngăn trượt và lật xe.
- **Hệ thống điều khiển tiện nghi:** Bao gồm điều hòa không khí, điều chỉnh ghế, gương, cửa sổ, đèn nội thất, thường kết nối qua mạng CAN tốc độ thấp (Low-Speed CAN).
- **Màn hình thông tin và giải trí (Infotainment):** Kết nối giữa bảng điều khiển trung tâm, màn hình, bộ định vị GPS, hệ thống âm thanh, và các nút điều khiển trên vô lăng.

Lợi ích nổi bật của CAN trong ngành ô tô

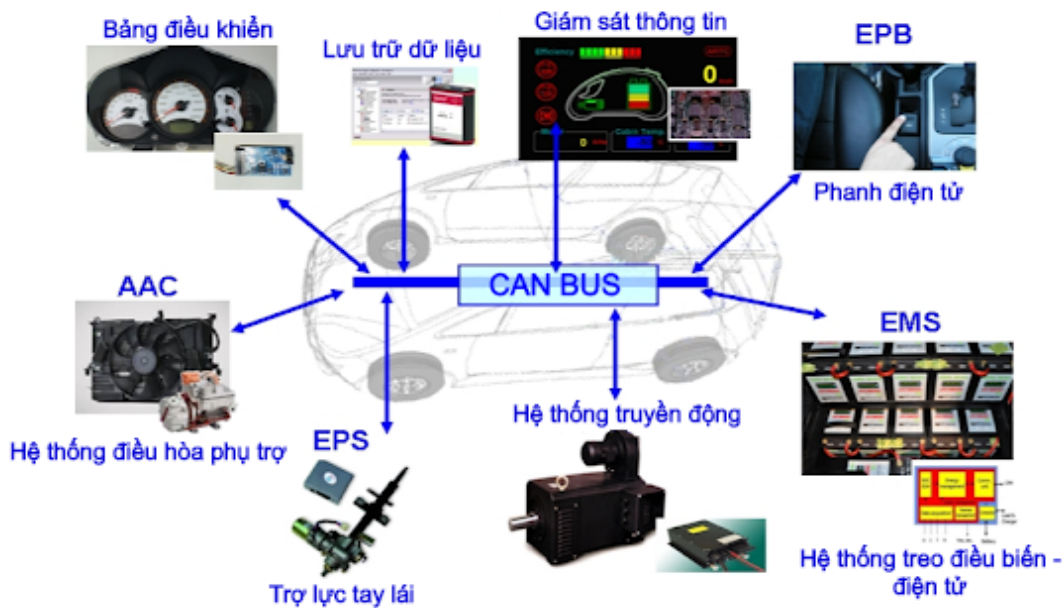
- **Giảm số lượng dây dẫn:** Trước đây, mỗi chức năng cần một đường dây riêng, gây rối rắm và nặng xe. CAN sử dụng một đường truyền chung, giúp tối ưu hóa thiết kế dây dẫn.
- **Tăng độ tin cậy:** Nhờ khả năng phát hiện và xử lý lỗi mạnh mẽ (CRC, ACK, Bit Monitoring...), hệ thống đảm bảo độ tin cậy cao ngay cả trong môi trường điện từ nhiễu loạn như khoang động cơ.
- **Khả năng mở rộng dễ dàng:** Việc bổ sung các mô-đun mới (như cảm biến, ECU phụ) không ảnh hưởng đến hoạt động của mạng hiện tại, do các node có thể giao tiếp độc lập và lọc thông điệp bằng ID.
- **Đáp ứng thời gian thực:** Đối với các hệ thống yêu cầu phản ứng nhanh (như phanh ABS, túi khí...), CAN cung cấp cơ chế **arbitration ưu tiên** dựa trên ID để đảm bảo thông điệp quan trọng được truyền đi trước.

Phân tầng mạng CAN trong ô tô

Trong thực tế, các hệ thống điện tử trong xe thường được chia thành các tầng mạng CAN khác nhau, tùy vào tốc độ và mức độ quan trọng:

- **High-Speed CAN (500 kbps – 1 Mbps):** Dùng cho các hệ thống an toàn và điều khiển động cơ, yêu cầu tốc độ cao và phản hồi nhanh.

- **Low-Speed / Fault-Tolerant CAN (125 kbps):** Dùng cho các chức năng thân xe như đèn, gương, khóa cửa, nơi yêu cầu tốc độ không quá cao nhưng độ ổn định quan trọng.
- **CAN FD (Flexible Data-rate):** Được tích hợp trong các xe mới để truyền lượng dữ liệu lớn hơn (như từ camera, radar) với tốc độ cao hơn và độ trễ thấp hơn.



Hình 5: Mô hình mạng CAN đơn giản trong hệ thống ô tô

Chương 3: Bảo mật trong hệ thống mạng CAN

I. Các vấn đề về bảo mật trong hệ thống mạng CAN

Mạng CAN (Controller Area Network) được thiết kế từ đầu nhằm tối ưu hoá cho tốc độ truyền thông và độ tin cậy trong các hệ thống nhúng, đặc biệt là ô tô. Tuy nhiên, thiết kế ban đầu của CAN không đề cập đến các yếu tố bảo mật, dẫn đến các lỗ hổng tiềm ẩn. Dưới đây là một số vấn đề bảo mật chính:

- **Không có cơ chế xác thực (Authentication):** Bất kỳ thiết bị nào được kết nối với bus CAN đều có thể gửi thông điệp. Mạng không có cách nào để xác minh nguồn gốc thông điệp.
- **Không có mã hoá (Encryption):** Dữ liệu trên bus CAN được truyền ở dạng rõ ràng (plaintext), cho phép kẻ tấn công dễ dàng phân tích và hiểu được nội dung.
- **Không có phân quyền truy cập (Access Control):** Tất cả các nút đều có quyền truy cập bình đẳng đến mạng, không có kiểm soát vai trò hay đặc quyền.
- **Dễ bị tấn công từ chối dịch vụ (Denial-of-Service - DoS):** Một thiết bị có thể chiếm quyền ưu tiên truy cập bằng cách gửi các frame có ID thấp liên tục, làm gián đoạn hoạt động của các thiết bị khác.
- **Khả năng nghe trộm (Eavesdropping):** Vì toàn bộ bus chia sẻ chung tín hiệu, nên mọi node đều có thể quan sát toàn bộ thông điệp, tạo điều kiện thuận lợi cho việc thu thập thông tin trái phép.
- **Tấn công giả mạo (Spoofing):** Một thiết bị độc hại có thể giả mạo ID của một thiết bị hợp lệ và gửi thông điệp sai lệch lên mạng.

II. Các phương pháp bảo mật trong hệ thống mạng CAN

Để khắc phục các vấn đề nêu trên, nhiều giải pháp bảo mật đã được đề xuất. Dưới đây là một số phương pháp phổ biến:

1. Mã hóa thông tin (Encryption)

Nguyên lý: Dữ liệu truyền trên bus CAN được mã hóa để ngăn chặn việc bị nghe lén (eavesdropping) hoặc chỉnh sửa trái phép. Nhờ đó, chỉ các thiết bị hợp lệ mới có thể giải mã và sử dụng thông điệp.

Cách thực hiện:

- **Mã hóa đối xứng (AES):** Tất cả các node chia sẻ một khóa bí mật chung. Mỗi thông điệp được mã hóa trước khi truyền, và chỉ có các node có khóa đúng mới giải mã được.
- **Mã hóa bất đối xứng (RSA):** Mỗi node có một cặp khóa công khai và bí mật. Node gửi sử dụng khóa công khai của node nhận để mã hóa, và node nhận sử dụng khóa bí mật để giải mã.

Ví dụ: Một node gửi dữ liệu D được mã hóa thành $E_K(D)$ bằng AES-128. Node nhận dùng cùng khóa K để giải mã trở lại thành D .

Ưu điểm:

- Đảm bảo tính riêng tư và an toàn của thông điệp.
- Ngăn chặn nghe lén và giả mạo nội dung.

Nhược điểm:

- Tốn tài nguyên xử lý và bộ nhớ (đặc biệt với RSA).
- Gây tăng độ trễ truyền thông nếu không tối ưu.
- Khó khăn trong việc quản lý khóa an toàn trong môi trường nhúng.

2. Xác thực thông điệp (Message Authentication)

Nguyên lý: Gắn thêm một mã xác thực (MAC - Message Authentication Code) vào mỗi thông điệp để kiểm tra nguồn gốc và tính toàn vẹn.

Cách thực hiện:

- Sử dụng hàm băm kết hợp với khóa bí mật để tạo mã xác thực, ví dụ: HMAC-SHA256.
- Node gửi tính giá trị MAC từ dữ liệu và gửi cùng trong payload.
- Node nhận tính lại MAC và so sánh với giá trị đã nhận. Nếu khớp, thông điệp hợp lệ.

Ví dụ: Với dữ liệu D và khóa K , node gửi tính $MAC = HMAC_K(D)$ rồi gửi (D, MAC) . Node nhận tính lại $HMAC_K(D)$ và so sánh.

Ưu điểm:

- Ngăn chặn giả mạo thông điệp.

- Phát hiện sửa đổi dữ liệu trong quá trình truyền.

Nhược điểm:

- Không bảo vệ chống phát lại (nên kết hợp với bộ đếm hoặc timestamp).
- Tăng kích thước payload (có thể vượt quá giới hạn 8 byte trong CAN 2.0).
- Tăng tải tính toán trên MCU.

3. Sử dụng bộ đếm tin nhắn đơn giản (Simple Message Counter)

Nguyên lý: Gắn một bộ đếm tăng dần vào mỗi thông điệp để đảm bảo tính "*tươi mới*" (freshness). Thiết bị nhận kiểm tra bộ đếm này để từ chối các thông điệp bị phát lại.

Cách thực hiện:

- Gắn một trường bộ đếm nhỏ (ví dụ: 8-bit hoặc 16-bit) vào thông điệp M_k . Mỗi lần gửi, bộ đếm được tăng lên.
- Thiết bị nhận lưu giá trị bộ đếm gần nhất nhận được cho mỗi ID hoặc phiên giao tiếp.
- Nếu bộ đếm trong thông điệp mới không lớn hơn giá trị đã lưu, thông điệp bị từ chối vì bị xem là bản sao hoặc lỗi thời.

Ví dụ: Nếu thông điệp trước đó có bộ đếm là 5, thông điệp tiếp theo hợp lệ phải có bộ đếm là 6 hoặc lớn hơn.

Ưu điểm:

- Cơ chế đơn giản, hiệu quả và dễ triển khai trên các vi điều khiển giới hạn tài nguyên.
- Giúp chống lại các cuộc tấn công phát lại đơn giản.

Nhược điểm:

- Không tự bảo vệ trước giả mạo hoặc nghe lén nếu không kết hợp với các cơ chế khác như MAC hoặc mã hoá.
- Yêu cầu thiết bị nhận phải lưu trạng thái bộ đếm cho từng nguồn gửi (có thể gây tốn RAM nếu hệ thống lớn).

4. Phát hiện xâm nhập (Intrusion Detection System - IDS)

Nguyên lý: Hệ thống IDS giám sát và phân tích luồng dữ liệu trên bus CAN để phát hiện các hành vi bất thường hoặc độc hại.

Cách thực hiện:

- Thu thập dữ liệu CAN và xác định hành vi bất thường dựa trên quy tắc (rule-based) hoặc học máy (machine learning).
- So sánh với mẫu hành vi bình thường để phát hiện bất thường như tần suất frame cao bất thường, hoặc gửi từ ID lạ.
- Gửi cảnh báo hoặc chặn giao tiếp nếu phát hiện nguy hiểm.

Ví dụ: Nếu một ID chỉ gửi 10 thông điệp/phút nhưng đột nhiên gửi 100 thông điệp trong vài giây, IDS sẽ báo động.

Ưu điểm:

- Không yêu cầu thay đổi giao thức CAN hiện có.
- Có thể phát hiện được các cuộc tấn công mới, chưa biết trước.

Nhược điểm:

- Có thể gây cảnh báo giả (false positive).
- Yêu cầu bộ xử lý mạnh hoặc vi điều khiển có khả năng xử lý tín hiệu cao.

5. Phân vùng mạng (Segmentation)

Nguyên lý: Chia hệ thống CAN thành nhiều vùng nhỏ để giới hạn ảnh hưởng khi xảy ra sự cố bảo mật. Các vùng giao tiếp với nhau thông qua thiết bị trung gian như gateway hoặc firewall.

Cách thực hiện:

- Thiết kế hệ thống gồm các vùng logic riêng biệt (ví dụ: động cơ, giải trí, phanh).
- Giao tiếp giữa các vùng được kiểm soát bởi gateway có chức năng xác thực và kiểm tra thông điệp.

Ví dụ: Vùng giải trí và điều khiển động cơ không trực tiếp kết nối, giúp hacker từ vùng giải trí không thể tác động tới ECU động cơ.

Ưu điểm:

- Hạn chế phạm vi tấn công khi một node bị chiếm quyền điều khiển.
- Dễ tích hợp với các cơ chế kiểm soát truy cập và lọc thông điệp.

Nhược điểm:

- Làm tăng độ phức tạp trong thiết kế hệ thống.
- Cần thêm phần cứng hoặc phần mềm gateway.

6. Chữ ký số (Digital Signatures)

Nguyên lý: Sử dụng chữ ký số để xác thực nguồn gốc và đảm bảo tính toàn vẹn của thông điệp, sử dụng cơ chế mật mã bất đối xứng.

Cách thực hiện:

- Node gửi tạo ra một chữ ký từ thông điệp và khóa bí mật.
- Node nhận sử dụng khóa công khai để xác minh chữ ký có hợp lệ không.

Ví dụ: Với thông điệp M , node gửi tạo chữ ký $Sig = Sign_{K_{priv}}(M)$, và gửi cùng với M . Node nhận dùng K_{pub} để kiểm tra $Verify_{K_{pub}}(M, Sig)$.

Ưu điểm:

- Xác minh mạnh mẽ và đáng tin cậy.
- Không cần chia sẻ khóa bí mật giữa các node.

Nhược điểm:

- Rất tốn tài nguyên và thời gian xử lý.
- Kích thước chữ ký lớn (thường > 64 byte) vượt giới hạn CAN 2.0.

7. Giao thức bảo mật lớp cao (Higher-layer Security Protocols)

Nguyên lý: Các giao thức lớp ứng dụng được xây dựng để cung cấp bảo mật bổ sung cho giao thức CAN gốc.

Cách thực hiện:

- **SecOC (Secure Onboard Communication):** Một module trong AUTOSAR cung cấp xác thực và kiểm tra tính toàn vẹn của thông điệp bằng cách thêm thông tin bảo mật vào message.
- **CANcrypt:** Một giao thức nhẹ, cung cấp mã hóa dòng dữ liệu, xác thực và đồng bộ hóa khóa tạm thời phù hợp với giới hạn của CAN 2.0.

Ưu điểm:

- Có thể triển khai trên nền hệ thống hiện tại.
- Cung cấp bảo mật toàn diện (xác thực, tính toàn vẹn, tươi mới).

Nhược điểm:

- Yêu cầu đồng bộ hóa thời gian hoặc trạng thái.
- Phức tạp trong việc triển khai và bảo trì.

Chương 4: CAN trong STM32F103

I. Giới thiệu

Vi điều khiển STM32F103C8T6 thuộc dòng STM32F1 của STMicroelectronics, sử dụng lõi ARM Cortex-M3 tốc độ tối đa 72 MHz, tích hợp nhiều ngoại vi mạnh mẽ, trong đó có bộ điều khiển CAN (Controller Area Network) theo chuẩn CAN 2.0A/B. Việc tích hợp giao tiếp CAN cho phép STM32F103C8T6 dễ dàng kết nối với các hệ thống mạng công nghiệp hoặc ô tô mà không cần chip chuyển đổi ngoài.

Giao tiếp CAN trong STM32F103C8T6 hỗ trợ:

- Khung dữ liệu chuẩn (Standard Frame - 11 bit ID)
- Khung dữ liệu mở rộng (Extended Frame - 29 bit ID)
- Dữ liệu tối đa 8 byte mỗi frame
- Chế độ gửi một lần hoặc lặp lại
- Tốc độ truyền lên đến 1 Mbps
- Bộ lọc phần cứng cho phép chỉ tiếp nhận những frame phù hợp, giúp giảm tải xử lý phần mềm

Vi điều khiển này thường được sử dụng trong các ứng dụng:

- Điều khiển động cơ và robot có nhiều nút giao tiếp
- Mạng CAN trong xe điện, xe tự hành
- Giao tiếp giữa nhiều vi điều khiển trong hệ thống phân tán
- Các hệ thống công nghiệp cần giao tiếp ổn định, chống nhiễu

Một điểm quan trọng khi sử dụng CAN trên STM32F103C8T6 là cần bổ sung thêm transceiver vật lý như MCP2551 để chuyển đổi mức tín hiệu CAN logic (TTL) sang mức điện áp chuẩn CAN bus (differential), giúp truyền dữ liệu trên đường dài và trong môi trường nhiễu cao.

Việc cấu hình CAN trên STM32F103C8T6 có thể thực hiện bằng:

- Thư viện HAL
- Truy cập trực tiếp thanh ghi để tối ưu hóa tốc độ và dung lượng bộ nhớ (cách này là cách mà nhóm đang lựa chọn để làm)

II. Các thanh ghi

1. Điều khiển & trạng thái tổng

- CAN1→MCR (Master Control)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														DBF	
														rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESET	Reserved							TTCM	ABOM	AWUM	NART	RFLM	TXFP	SLEEP	INRQ
rs								rw	rw	rw	rw	rw	rw	rw	rw

- INRQ: Initialization Request: yêu cầu vào chế độ khởi tạo (1 = vào init)
- SLEEP: Sleep Mode Request
- ABOM: Automatic Bus-Off Management (1 = tự động thoát bus-off)

- CAN1→MSR (Master Status)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved.				RX	SAMP	RXM	TXM	Reserved				SLAKI	WKUI	ERRI	SLAK
				r	r	r	r					rc_w1	rc_w1	rc_w1	INAK

- INAK: Initialization Acknowledge: xác nhận đã vào init mode
- SLAK: Sleep Acknowledge
- ERRI: Error Interrupt Flag

- CAN1→ESR (Error Status)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REC[7:0]								TEC[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									LEC[2:0]			Res.	BOFF	EPVF	EWGF
									rw	rw	rw		r	r	r

- EWGF: Error Warning
- EPVF: Error Passive
- BOFF: Bus Off (CAN bị lỗi nghiêm trọng, ngừng hoạt động)

2. Định thời Bit

- CAN1→BTR (Bit Timing)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SILM	LBKM	Reserved				SJW[1:0]		Res.	TS2[2:0]			TS1[3:0]			
rw	rw					rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						BRP[9:0]									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- BRP[9:0]: Baud Rate Prescaler
- TS1[3:0]: Time Segment 1
- TS2[2:0]: Time Segment 2
- SJW[1:0]: Synchronization Jump Width
- SILM: Silent mode
- LBKM: Loopback mode

3. Gửi (TX)

- CAN1→TSR (Transmit Status)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOW2	LOW1	LOW0	TME2	TME1	TME0	CODE[1:0]		ABRQ2	Reserved			TERR2	ALST2	TXOK2	RQCP2
r	r	r	r	r	r	r	r	rs				rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRQ1	Reserved Res.			TERR1	ALST1	TXOK1	RQCP1	ABRQ0	Reserved			TERR0	ALST0	TXOK0	RQCP0
rs				rc_w1	rc_w1	rc_w1	rc_w1	rs				rc_w1	rc_w1	rc_w1	rc_w1

- Kiểm tra mailbox rồi, xoá cờ TXOK/ALST/TERR, phát hiện timeout

- CAN1→sTxMailBox[0].TIR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
STID[10:0]/EXID[28:18]											EXID[17:13]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXID[12:0]												IDE	RTR	TXRQ	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- TXRQ: Transmission Request (gửi gói tin nếu đặt bit này)
- STID[10:0]: Standard ID
- EXID[18:0]: Extended ID
- IDE: Identifier Extension (0 = standard, 1 = extended)
- RTR: Remote Transmission Request (0 = data frame)

- CAN1→sTxMailBox[0].TDTR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIME[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TGT	Reserved				DLC[3:0]		
								rw					rw	rw	rw

– DLC[3:0]: Data Length Code (số byte dữ liệu: 0–8)

- CAN1→sTxMailBox[0].TDLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA3[7:0]								DATA2[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA1[7:0]								DATA0[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

– Chứa dữ liệu gửi 4 byte thấp

- CAN1→sTxMailBox[0].TDHR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA7[7:0]								DATA6[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA5[7:0]								DATA4[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

– Chứa dữ liệu gửi 4 byte cao

4. Nhận (RX)

- CAN1→RF0R (FIFO 0)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										RFOM0	FOVR0	FULL0	Res.	FMP0[1:0]	
										rs	rc_w1	rc_w1		r	r

– FMP0[1:0]: FIFO message pending (số gói chưa đọc)

– RFOM0: Release FIFO Output Mailbox (xóa gói tin đã đọc)

- CAN1→sFIFOMailBox[0].RIR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
STID[10:0]/EXID[28:18]											EXID[17:13]				
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXID[12:0]													IDE	RTR	Res.
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	

– Chứa ID và kiểu frame

- CAN1→sFIFOMailBox[0].RDTR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIME[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMI[7:0]								Reserved				DLC[3:0]			
r	r	r	r	r	r	r	r					r	r	r	r

– Số byte (DLC)

- CAN1→sFIFOMailBox[0].RDLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA3[7:0]								DATA2[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA1[7:0]								DATA0[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

– Chứa dữ liệu nhận 4 byte thấp

- CAN1→sFIFOMailBox[0].RDHR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA7[7:0]								DATA6[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA5[7:0]								DATA4[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

– Chứa dữ liệu nhận 4 byte cao

5. Bộ lọc

- CAN1→FMR (Filter Master)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		CAN2SB[5:0]						Reserved						FINIT	
		rw	rw	rw	rw	rw	rw							rw	

– FINIT: Filter Init Mode

• CAN1→FM1R

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FBM27	FBM26	FBM25	FBM24	FBM23	FBM22	FBM21	FBM20	FBM19	FBM18	FBM17	FBM16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FBM15	FBM14	FBM13	FBM12	FBM11	FBM10	FBM9	FBM8	FBM7	FBM6	FBM5	FBM4	FBM3	FBM2	FBM1	FBM0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

– FM1R[x] = 0: Mask mode (ID & MASK)

– FM1R[x] = 1: List mode (so sánh trực tiếp với nhiều ID)

• CAN1→FS1R

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FSC27	FSC26	FSC25	FSC24	FSC23	FSC22	FSC21	FSC20	FSC19	FSC18	FSC17	FSC16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FSC15	FSC14	FSC13	FSC12	FSC11	FSC10	FSC9	FSC8	FSC7	FSC6	FSC5	FSC4	FSC3	FSC2	FSC1	FSC0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

– FS1R[x] = 1: Filter bank x dùng 32-bit mode (1 filter = 1 ID/mask 32-bit)

– FS1R[x] = 0: Filter bank x dùng 16-bit mode (2 filter = 2 ID/mask 16-bit)

• CAN1→FFA1R

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FFA27	FFA26	FFA25	FFA24	FFA23	FFA22	FFA21	FFA20	FFA19	FFA18	FFA17	FFA16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FFA15	FFA14	FFA13	FFA12	FFA11	FFA10	FFA9	FFA8	FFA7	FFA6	FFA5	FFA4	FFA3	FFA2	FFA1	FFA0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

– FFA0: 0: filter 0 vào FIFO0, 1: filter 0 vào FIFO1

– FFA1: 0: filter 1 vào FIFO0, 1: filter 1 vào FIFO1

– ...

– FFA27: 0: filter 27 vào FIFO0, 1: filter 27 vào FIFO1

• CAN1→FA1R

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				FACT27	FACT26	FACT25	FACT24	FACT23	FACT22	FACT21	FACT20	FACT19	FACT18	FACT17	FACT16
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FACT15	FACT14	FACT13	FACT12	FACT11	FACT10	FACT9	FACT8	FACT7	FACT6	FACT5	FACT4	FACT3	FACT2	FACT1	FACT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- FA1R[x] = 1: Filter bank x được kích hoạt
- FA1R[x] = 0: Filter bank x bị vô hiệu hóa

- CAN1→sFilterRegister[0].FR1/FR2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FB31	FB30	FB29	FB28	FB27	FB26	FB25	FB24	FB23	FB22	FB21	FB20	FB19	FB18	FB17	FB16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FB15	FB14	FB13	FB12	FB11	FB10	FB9	FB8	FB7	FB6	FB5	FB4	FB3	FB2	FB1	FB0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- FR1: Chứa ID cần so sánh đầu tiên (ID hoặc ID1 trong list mode)
- FR2: Chứa giá trị MASK (nếu dùng mask mode), hoặc ID thứ 2 (nếu list mode)

6. Ngắt

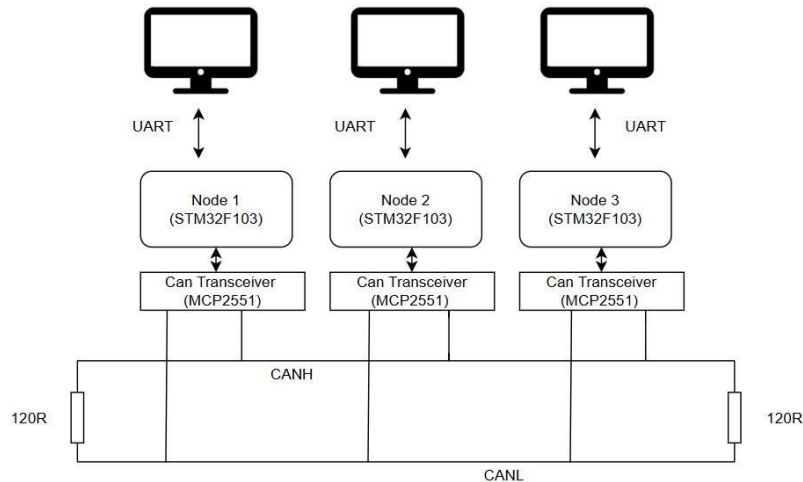
- CAN1→IER (Interrupt Enable)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														SLKIE	WKUIE
														rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ERRIE	Reserved				LEC IE	BOF IE	EPV IE	EWG IE	Res.	FOV IE1	FF IE1	FMP IE1	FOV IE0	FF IE0	FMP IE0
rw					rw	rw	rw	rw		rw	rw	rw	rw	rw	rw

- TMEIE: Cho phép ngắt khi mailbox truyền trống – dùng để truyền CAN bằng ngắt.
- FMPIE0: Cho phép ngắt khi nhận được dữ liệu vào FIFO0 – phổ biến nhất khi xử lý CAN RX.
- ERRIE: Cho phép ngắt khi có bất kỳ lỗi CAN nào xảy ra (bus-off, passive, CRC...)

Chương 5: Thiết kế hệ thống

I. Hệ thống phần cứng



Hình 6: Sơ đồ tổng quát hệ thống phần cứng.

Mô tả hệ thống:

- Các node sẽ là các vi điều khiển STM32F103 : các vi điều khiển này có chức năng giao tiếp với các Node khác bằng giao thức Can. Các Node sẽ nhận được yêu cầu và gửi thông tin lên máy tính thông qua giao thức Uart.
- Can Transceiver : là modun MCP2551 , modun này có tác dụng chuyển đổi tín hiệu từ CRX và TRX của vi điều khiển thành tín hiệu vi sai lên các đường CANH VÀ CANL.
- 2 con trở 120R có tác dụng giảm phản xạ tín hiệu, định ,mức trở kháng cho đường truyền và ổn định tín hiệu trên đường truyền.

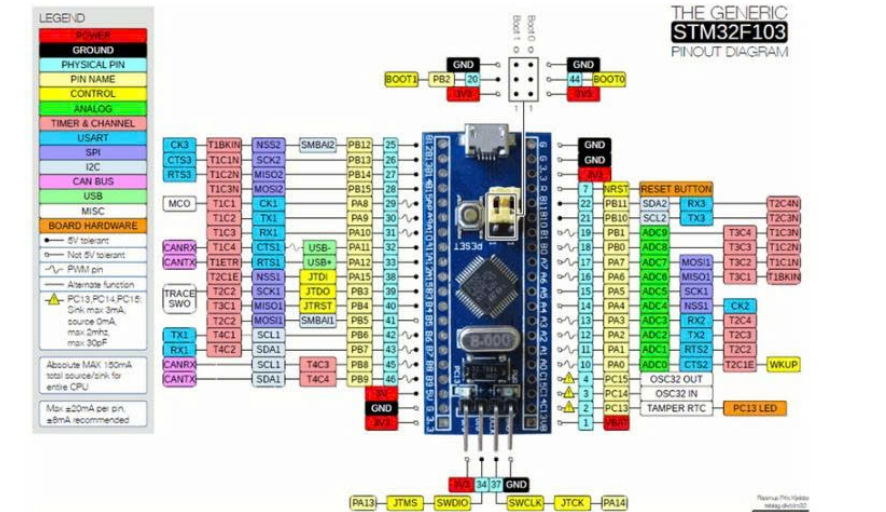
1. Linh kiện sử dụng

STM32F103C8T6

STM32F103C8T6 là vi điều khiển 32-bit của hãng STMicroelectronics thuộc dòng ARM Cortex-M3. Đây là một lựa chọn phổ biến trong các dự án nhúng nhờ vào giá thành rẻ, hiệu năng ổn và tài liệu hỗ trợ phong phú.

- **Vi xử lý:** ARM Cortex-M3, tốc độ xung nhịp tối đa 72 MHz.

- **Bộ nhớ:** 64 KB Flash, 20 KB SRAM.
- **Giao tiếp:** Hỗ trợ nhiều chuẩn giao tiếp như UART, SPI, I2C, USB, và đặc biệt là **CAN Bus**.
- **Nguồn:** Hoạt động ở 3.3V (vẫn tương thích mức 5V với một số chân).
- **Kích thước nhỏ gọn:** Dạng module có thể cắm trực tiếp vào breadboard.



Hình 7: STM32F103

Các chân **CAN RX** và **CAN TX** tương ứng lần lượt là **PA11** và **PA12**, được kết nối tới module transceiver CAN như MCP2551.

MCP2551 - CAN Transceiver

MCP2551 là một mạch tích hợp transceiver vật lý cho giao tiếp CAN (Controller Area Network), đảm nhiệm việc chuyển đổi tín hiệu giữa vi điều khiển và bus CAN.

- **VCC:** Cấp nguồn hoạt động, thường là 5V.
- **CTX (TXD):** Chân truyền dữ liệu từ vi điều khiển (kết nối với PA12 của STM32).
- **CRX (RXD):** Chân nhận dữ liệu từ bus CAN (kết nối với PA11 của STM32).
- **CANH và CANL:** Hai dây tín hiệu CAN chuẩn dùng để giao tiếp với các node CAN khác.
- **Tốc độ:** Hỗ trợ tốc độ lên đến 1 Mbps.
- **Chức năng:** Cách ly giao tiếp vật lý giữa vi điều khiển và bus CAN giúp tránh hư hỏng khi có nhiễu hoặc xung điện.



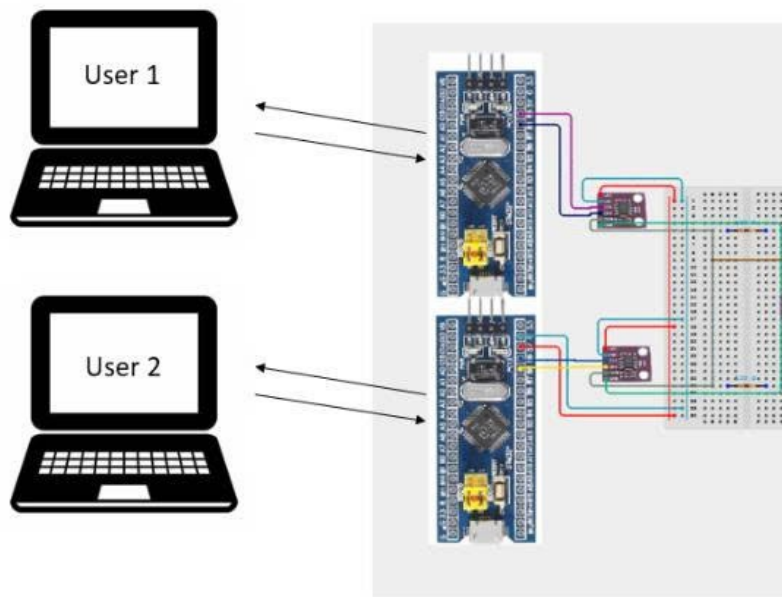
Hình 8: MCP2551

Sơ đồ kết nối cơ bản giữa STM32 và MCP2551:

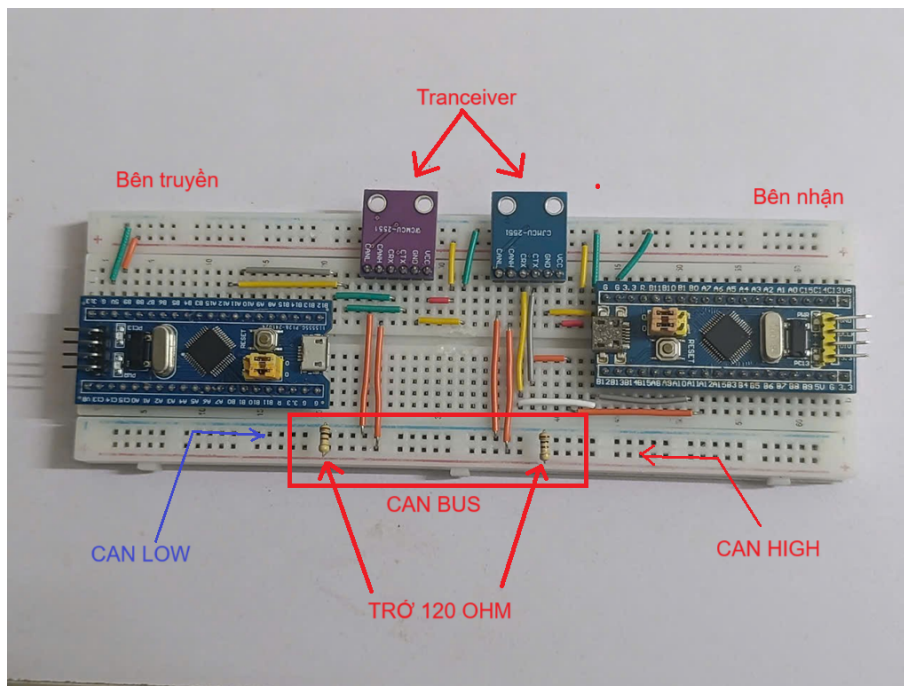
- PA12 (CAN_TX) → CTX (TXD)
- PA11 (CAN_RX) → CRX (RXD)
- CANH ↔ Dây CAN H
- CANL ↔ Dây CAN L
- VCC → 5V; GND → GND

2. Sơ đồ mạch

Sơ đồ mạch truyền Can 2 chiều



Hình 9: Can 2 chiều



Hình 10: Mạch thực tế truyền nhận CAN.

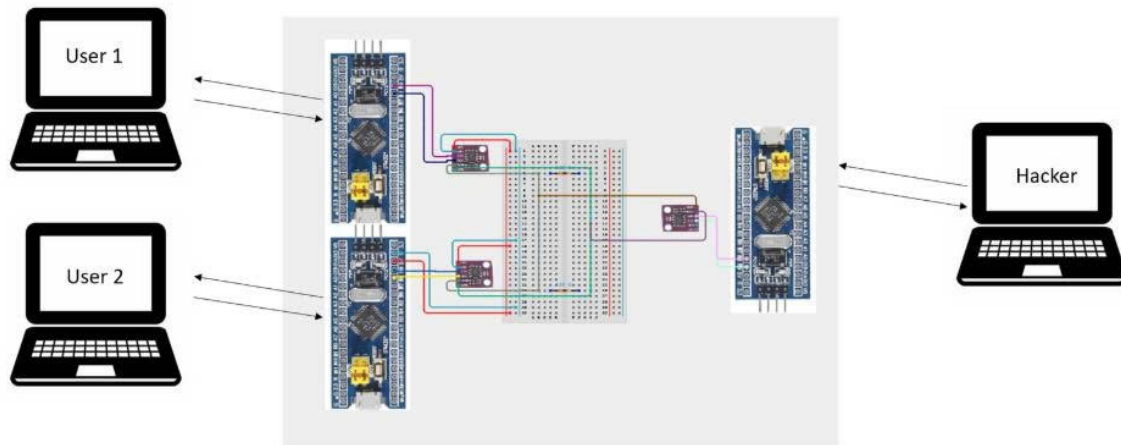
Trong hệ thống truyền nhận của mạng CAN cơ bản gồm 2 nút - tương ứng với 2 vi điều khiển đóng vai trò truyền - nhận. Nhóm chúng tôi sử dụng 2 vi điều khiển STM32F103C8T6 làm 2 node giao tiếp trong mạng CAN. Cấu hình các chân PA11 (CAN_RX) và PA12 (CAN_TX) để kết nối với Tranceiver CMP2551.

Tranceiver CMP2551 chuyển tín hiệu nhận được từ vi điều khiển rồi chuyển tiếp

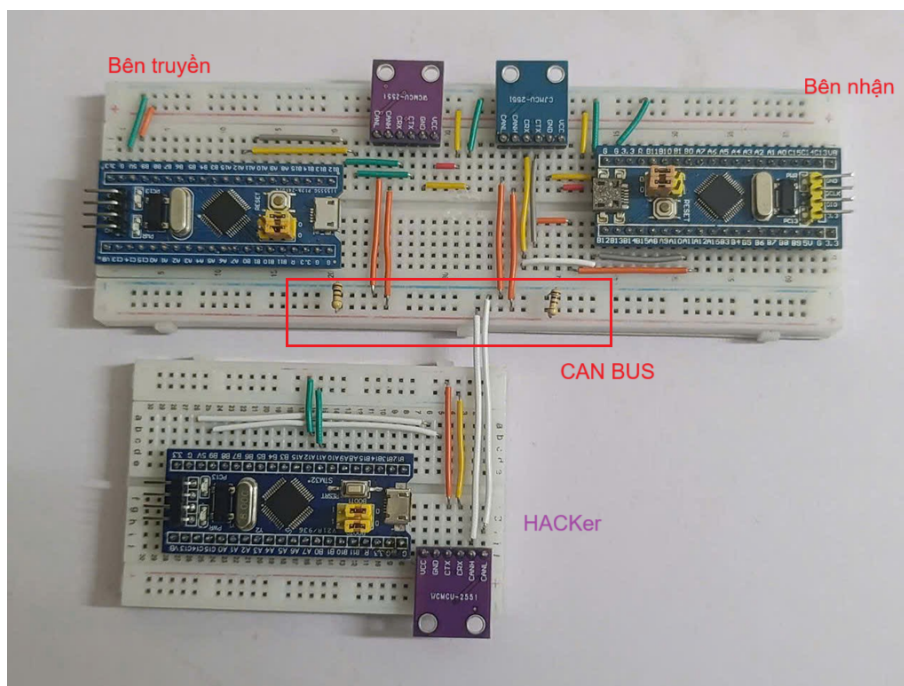
thành tín hiệu vi sai trên CAN bus.

Do độ dài của BUS ngắn nên CAN bus không cần là cặp dây xoắn song song. 2 đầu CAN bus lắp thêm điện trở 120 Ω để phối hợp trở kháng với đường dây và điện trở tại các nút, giúp cho tín hiệu không bị phản xạ lại.

Sơ đồ mạch truyền Can có thêm hacker



Hình 11: Sơ đồ mạch truyền Can có thêm hacker



Hình 12: Mạch thực tế truyền Can có thêm hacker

Để thực hiện 1 hệ thống mạng CAN có bảo mật, chúng tôi thêm 1 node thứ 3 vào

để mô phỏng làm 1 hacker muốn tấn công vào hệ thống.

Cấu hình của node thứ 3 cũng tương tự với 2 node truyền nhận, node được nối thêm vào CAN bus để để đọc lên dữ liệu và thực hiện tấn công mạng CAN bằng cách Replay Attack những tín hiệu đã truyền trước đó. Mục đích của việc này là làm cho bên nhận tín hiệu nhầm lẫn và thực hiện các yêu cầu không mong muốn.

II. Hệ thống phần mềm

Trong hệ thống truyền thông CAN, phần mềm đóng vai trò quan trọng trong việc xử lý, hiển thị và bảo vệ dữ liệu truyền nhận. Dưới đây là ba giao diện phần mềm được xây dựng để minh họa hoạt động truyền nhận dữ liệu CAN, cũng như mô phỏng các kịch bản có và không có bảo vệ, và cả tấn công mạng.

1. Giao diện truyền nhận CAN chưa có bảo vệ

Giao diện ban đầu được thiết kế cho mục đích truyền và nhận dữ liệu CAN cơ bản giữa các node. Tuy nhiên, hệ thống này chưa tích hợp các cơ chế bảo vệ, do đó dễ bị can thiệp hoặc làm nhiễu từ các thiết bị không hợp lệ trong cùng mạng CAN.

The screenshot displays the 'CAN Cybersecurity' web interface. It is divided into four main sections:

- UART Config:** Includes fields for 'Port' (set to COM1) and 'Baudrate' (set to 115200), with a red 'Connect' button.
- CAN Config:** Includes fields for 'CAN ID' (set to 01A3), 'Data' (empty), and 'Cyclic' (checked), with a blue 'Add' button.
- Transmit:** A table showing transmitted messages.

CAN ID	Model	Data	Description	Direction	Cyclic	Send	Edit	Delete
01A3	Standard	Winer	Temperature	Tx	0	Send	Edit	Delete
1COFFEE0	Extended	Loser	Music	Tx	1000	Send	Edit	Delete
02BC	Standard	30m/s	Velocity	Tx	5000	Send	Edit	Delete
1ABCD0F	Extended	HPBD	Blink LED	Tx	0	Send	Edit	Delete
- Receive:** A table showing received messages.

Timestamp	CAN ID	Model	Data	Description	Direction	Delete
2025-06-09 16:44:07	02BC	Standard	90m/s	Velocity	Rx	Delete
2025-06-09 16:44:05	02BC	Standard	90m/s	Velocity	Rx	Delete
2025-06-09 16:44:03	02BC	Standard	90m/s	Velocity	Rx	Delete
2025-06-09 16:43:59	1COFFEE0	Extended	haha	Music	Rx	Delete
2025-06-09 16:43:57	03FS	Standard	30%	Battery Percent	Rx	Delete
2025-06-09 16:43:55	03FS	Standard	30%	Battery Percent	Rx	Delete
2025-06-09 16:43:53	03FS	Standard	30%	Battery Percent	Rx	Delete
2025-06-09 16:43:39	1COFFEE0	Extended	haha	Music	Rx	Delete

Hình 13: Giao diện truyền nhận CAN chưa có bảo vệ

Giao diện chia thành 4 phần chính:

- **UART Config:** Cho phép người dùng chọn cổng COM và tốc độ Baudrate (ví dụ: 115200) để kết nối với thiết bị phần cứng. Khi nhấn *Connect*, một chấm tròn màu đỏ sẽ chuyển sang màu khác để biểu thị trạng thái kết nối thành công.

- **CAN Config:** Cho phép nhập các thông số của khung truyền như *CAN ID*, dữ liệu (*Data*), và chu kỳ lặp (*Cyclic*) được lấy từ database được tạo sẵn nếu muốn gửi định kỳ. Nhấn nút *Add* để thêm khung vào bảng danh sách truyền.
- **Transmit Table:** Hiển thị các khung CAN đã cấu hình để truyền, với thông tin như:
 - *CAN ID*: định danh gói tin.
 - *Model*: loại frame (Standard hoặc Extended).
 - *Data, Description*: dữ liệu và mô tả chức năng.
 - *Direction*: hướng truyền (Tx).
 - *Cyclic*: chu kỳ gửi định kỳ (ms).
 - Các nút chức năng: *Send, Edit, Delete*.
- **Receive Table:** Ghi nhận và hiển thị các gói tin CAN nhận được từ bus, bao gồm:
 - *Timestamp*: thời gian nhận.
 - *CAN ID, Model, Data*.
 - *Description, Direction* (Rx).
 - Nút *Delete* để xóa dòng dữ liệu.

Hạn chế của giao diện chưa bảo vệ:

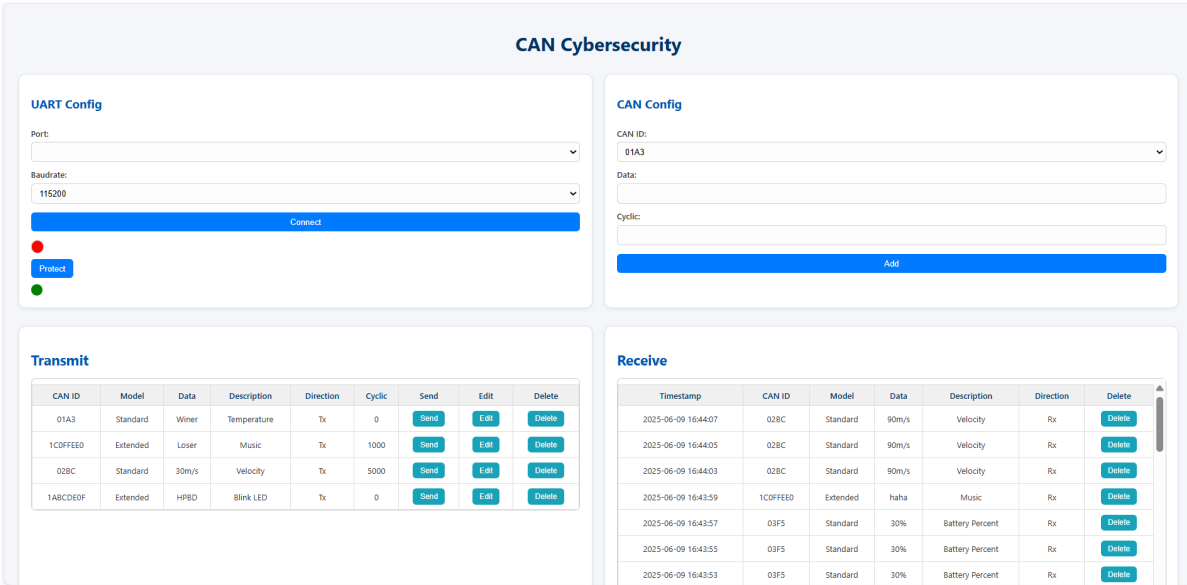
- Không có xác thực nguồn gửi (CAN ID có thể bị giả mạo).
- Không có lọc nội dung hoặc kiểm tra hợp lệ.
- Không mã hóa dữ liệu, dễ bị đọc trộm hoặc sửa đổi.
- Không có cảnh báo khi có hành vi bất thường (flooding, gửi liên tục).

2. Giao diện truyền nhận CAN có bảo vệ (Protected CAN)

Giao diện này được nâng cấp từ phiên bản cơ bản, với tính năng phát hiện replay attack bởi bên thứ 3.

Việc tích hợp lớp bảo vệ giúp đảm bảo tính toàn vẹn và an toàn cho dữ liệu truyền trong mạng CAN.

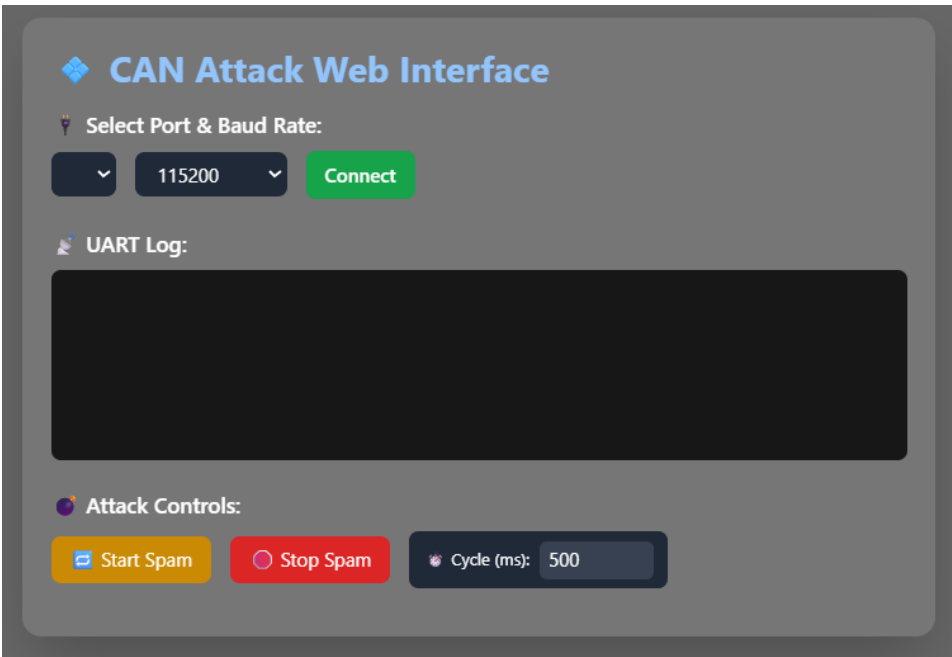
Giao diện sẽ tương tự phần 1 nhưng có thêm nút protect, nếu không có protect, đèn led hiện màu đỏ tức chưa bật chống replay attack, nếu đèn led màu xanh lá cây tức là đã bật protect, sẽ tránh được sự tấn công của bên thứ 3.



Hình 14: Giao diện truyền nhận CAN có bảo vệ

3. Giao diện tấn công mạng CAN (CAN Attack Web Interface)

Giao diện tấn công này được thiết kế nhằm mô phỏng các hành vi tấn công phổ biến trong mạng CAN như spam frame, gây nghẽn mạng (DoS - Denial of Service), hoặc kiểm tra khả năng phản hồi của hệ thống mục tiêu. Việc xây dựng và sử dụng giao diện này giúp minh họa rõ hơn các lỗ hổng khi hệ thống thiếu cơ chế bảo vệ phù hợp.



Hình 15: Giao diện mô phỏng tấn công mạng CAN

Các thành phần chính:

- **Select Port & Baud Rate:** Chọn cổng COM và tốc độ truyền UART để kết nối với thiết bị tấn công (ví dụ: một Node giả mạo hoặc công cụ mô phỏng tấn công).
- **UART Log:** Hiển thị các bản ghi từ thiết bị trong quá trình tấn công, là dữ liệu nhận được từ CAN Bus khi xâm nhập đường truyền.
- **Attack Controls:**
 - *Start Spam:* Bắt đầu gửi liên tục các frame CAN giả mạo với chu kỳ cấu hình.
 - *Stop Spam:* Dừng việc gửi spam.
 - *Cycle (ms):* Khoảng thời gian giữa các frame spam, có thể điều chỉnh để kiểm tra tác động ở nhiều mức độ khác nhau.

Mục đích và nguy cơ:

- **Mô phỏng tấn công:** Gửi liên tục gói tin nhận được lặp lại với chu kỳ ngắn khiến bên nhận nhận thừa thông tin.
- **Phát hiện lỗi hỏng giao thức:** Thử nghiệm để tìm ra các tình huống không xử lý đúng.

Việc xây dựng và sử dụng giao diện tấn công này phục vụ mục tiêu học tập và đánh giá an toàn mạng CAN, không được sử dụng cho mục đích phá hoại hệ thống thực tế.

4. Cơ sở dữ liệu

Hệ thống được xây dựng cơ sở dữ liệu để thuận tiện cho việc truy xuất và lưu trữ các gói tin truyền nhận. Chúng tôi sử dụng MySQL để làm cơ sở dữ liệu, các website truyền và nhận sẽ liên kết trực tiếp với cơ sở dữ liệu để truy xuất các gói tin cũng như lưu trữ lại các gói tin nhận được.

Cơ sở dữ liệu bao gồm 3 bảng chính: CAN, Transmit và Receive. Trong đó:

Bảng CAN:

Bảng này được sử dụng để lưu trữ các miêu tả của gói tin. Gồm các mô tả ứng với từng id riêng biệt. Mục đích của bảng này là xác định rõ mục đích của từng gói tin trong hệ thống. Tất cả các phần tử trong hệ thống đều có thể truy cập bảng này để xác định được mô tả của gói tin.

Ví dụ về bảng dữ liệu các mô tả như sau:

id	model	can_id	description
1	Standard	01A3	Temperature
2	Standard	02BC	Velocity
3	Extended	1ABCDE0F	Blink LED
4	Extended	1C0FFEE0	Music
5	Standard	03F5	Battery Percent
6	Standard	01F5	Engine Status

Từ bảng có thể thấy, với các gói tin có can_id là 01A3 thì gói tin đó có nội dung thể hiện về Temperature. Hay như gói tin có can_id là 03F5 thì nội dung của nó sẽ là về Battery Percent.

Các can_id được định sẵn trong hệ thống, các phần tử muốn gửi tin lên CAN bus chỉ có thể sử dụng các can_id trong này nhằm tạo sự đồng bộ cũng như tránh sử dụng các gói tin có can_id không được xác định.

Bảng Transmit:

Bảng Transmit dùng để lưu trữ các gói tin được tạo để truyền đi. Các bước thêm vào cơ sở dữ liệu như ảnh sau:

The image shows a software interface for a logic analyzer. It has four main panels: UART Config, CAN Config, Transmit, and Receive. The CAN Config panel has a red box around the 'CAN ID' dropdown (set to '1ABCDE0F'), the 'Data' input field, and the 'Cyclic' checkbox. Below these is a blue 'Add' button, with a red arrow pointing to it from the text '2. thêm vào cơ sở dữ liệu'. The Transmit panel has a red box around a table, with a red arrow pointing to it from the text '3. Hiện thị data từ cơ sở dữ liệu'. The table has columns: CAN ID, Model, Data, Description, Direction, Cyclic, Send, Edit, and Delete. It contains three rows of data. The Receive panel is empty.

Hình 16: Logic analyzer

Cơ sở dữ liệu của bảng Transmit gồm các nội dung như bảng dưới đây:

model	can_id	data	description	direction	cyclic
Extended	1ABCDE0F	2-times	Blink LED	Tx	1000
Standard	01A3	30C	Temperature	Tx	1000
Standard	035F	30%	Battery Percent	Tx	0
Extended	1C0FFEE0	bebe	Music	Tx	0

Trong đó nội dung của từng trường là:

- **model:** Dạng id của gói tin, Standard hoặc Extended.
- **can_id:** id của gói từng gói tin.
- **data:** nội dung cần gửi đi.
- **description:** mô tả của của gói tin đó. Nội dung này sẽ không được gửi đi mà sẽ được lấy từ trong bảng CAN bên trên.
- **direction:** Hướng của gói tin, 'Tx' thể hiện gói tin này được truyền đi.
- **cyclic:** chu kỳ truyền của gói tin, xác định theo ms. cyclic = 0 tức là gói tin được truyền đi 1 lần sau mỗi lần nhấn gửi trên giao diện, còn lớn hơn 0 thì sẽ được gửi liên tục sau khi nhấn gửi.

Bảng Receive:

Bảng này dùng để lưu các gói tin nhận được, về cơ bản các trường trong cơ sở dữ liệu cũng tương tự như bảng transmit, chỉ thay trường cyclic thành trường timestamp để thể hiện thời gian nhận được gói tin đó.

Dưới đây là cơ sở dữ liệu của bảng Receive:

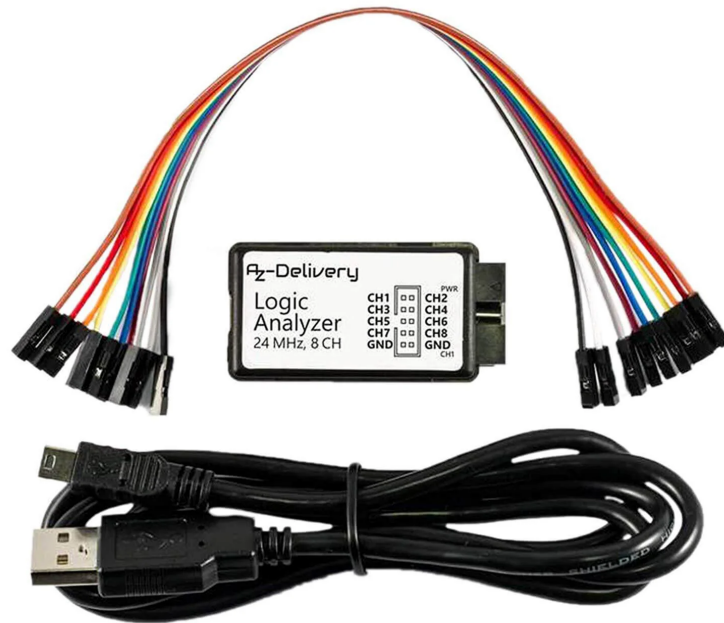
model	can_id	data	description	direction	timestamp
-------	--------	------	-------------	-----------	-----------

Trường timestamp sẽ lưu lại giá trị thời gian mà có dữ liệu mới được thêm vào cơ sở dữ liệu. Trường timestamp này dùng để ta theo dõi liên tục các gói tin nhận được trong quá trình hoạt động. Trong một số hệ thống, họ cũng sử dụng timestamp để nhận diện, phát hiện các cuộc tấn công của hacker.

III. Hệ thống đo đạc

1. Logic Analyzer 8 Kênh

Logic Analyzer (bộ phân tích logic) là thiết bị dùng để ghi nhận và hiển thị các tín hiệu số theo thời gian. Đây là công cụ không thể thiếu trong quá trình kiểm thử, phân tích giao tiếp số (như UART, SPI, I2C, CAN...) giữa các vi điều khiển và thiết bị ngoại vi.



Hình 17: Logic analyzer

Đặc điểm chính:

- **Số kênh:** 8 kênh phân tích tín hiệu số (CH0 đến CH7).
- **Tốc độ lấy mẫu:** Thông thường từ 1 24 MHz (tùy cấu hình), đủ dùng để bắt các tín hiệu tốc độ vừa và thấp như UART, I2C, SPI.
- **Giao tiếp:** Kết nối với máy tính qua USB, dùng phần mềm như **PulseView** (thuộc bộ công cụ *Sigrok*).
- **Điện áp hoạt động:** Tương thích mức tín hiệu 3.3V và 5V.
- **Dạng tín hiệu hỗ trợ:** Phân tích tín hiệu số như: UART, I2C, SPI, CAN, PWM, v.v.

Cách sử dụng:

- a. Kết nối các dây logic analyzer tới các chân tín hiệu của mạch cần phân tích (CH0-CH7, GND).
- b. Cắm module vào máy tính qua cổng USB.
- c. Mở phần mềm PulseView, chọn đúng thiết bị và tốc độ lấy mẫu.
- d. Bấm "Start Capture" để bắt đầu thu tín hiệu.

Ưu điểm:

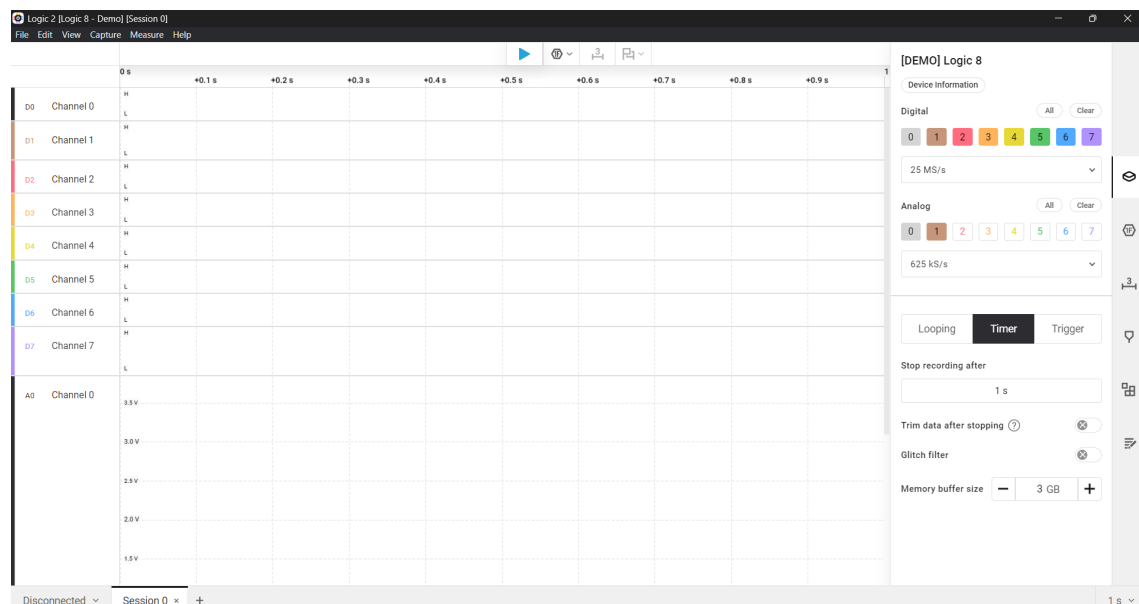
- Giá thành rẻ, dễ sử dụng.
- Giao diện phần mềm trực quan, hỗ trợ giải mã các giao thức phổ biến.
- Có thể lưu dữ liệu phân tích để tra cứu lại.

Ứng dụng:

- Gỡ lỗi truyền thông UART, SPI, I2C, CAN.
- Kiểm tra độ ổn định tín hiệu PWM, đo tần số, độ rộng xung.
- Theo dõi luồng dữ liệu giữa các vi điều khiển.

2. Logic 2

Logic 2 là phần mềm chính thức của hãng Saleae, dùng để điều khiển và thu thập dữ liệu từ các thiết bị phân tích logic (Logic Analyzer) do Saleae sản xuất. Đây là công cụ cực kỳ mạnh cho kỹ sư nhúng, lập trình viên firmware/hardware và các kỹ sư kiểm thử phần cứng.



Hình 18: Logic analyzer

Phần mềm hỗ trợ cả phân tích tín hiệu số (digital) lẫn tín hiệu analog, với giao diện trực quan, dễ dùng và rất mạnh mẽ trong phân tích giao thức truyền thông.

IV. Nguyên lý hoạt động của hệ thống

Hệ thống mô phỏng mạng truyền thông CAN (Controller Area Network) trong xe ô tô, với mục tiêu chính là xây dựng một mô hình giao tiếp hai chiều giữa các node hợp lệ, đồng thời phát hiện và ngăn chặn các hành vi tấn công mạng từ các thiết bị xâm nhập trái phép. Kiến trúc hệ thống được thiết kế gồm ba thành phần chính:

- **Node truyền hợp lệ (Trusted Transmitter):** Đóng vai trò gửi dữ liệu CAN hợp lệ đến các node khác trong mạng.
- **Node nhận hợp lệ (Trusted Receiver):** Tiếp nhận và xử lý các frame CAN đến, kiểm tra tính toàn vẹn và xác thực trước khi hiển thị.
- **Node tấn công (Malicious Node):** Giả lập tình huống tấn công mạng replay attack với mục đích làm gián đoạn hoặc thay đổi nội dung giao tiếp.

Hệ thống được chia thành 3 giai đoạn vận hành chính:

1. Giai đoạn giao tiếp CAN cơ bản (Chưa có bảo vệ)

Trong giai đoạn này, hệ thống thực hiện truyền và nhận dữ liệu CAN giữa các node hợp lệ mà không có bất kỳ cơ chế bảo mật nào. Mỗi frame CAN được gửi gồm có:

- **Arbitration ID:** Dùng để phân biệt loại dữ liệu và mức độ ưu tiên.
- **Data field:** Chứa nội dung thực của dữ liệu (ví dụ: tốc độ xe, góc đánh lái,...).
- **CRC field:** Mã kiểm tra lỗi truyền dẫn.

Ở chế độ này, bất kỳ node nào tham gia mạng đều có thể gửi frame CAN, kể cả các thiết bị không được xác thực. Do đó, hệ thống dễ bị tấn công bởi Replay Attack: Gửi lại các frame trước đó một cách lặp đi lặp lại để gây nhầm lẫn hoặc gây nghẽn.

2. Giai đoạn tấn công mạng CAN

Trong hệ thống này chúng em thiết kế để chống lại Replay attack (tấn công lặp lại liên tục).

Tấn công lặp lại là một dạng tấn công trong đó kẻ tấn công ghi lại các gói tin hợp lệ được truyền qua mạng CAN, sau đó phát lại chúng tại một thời điểm khác nhằm điều khiển hành vi của hệ thống. Do CAN không xác minh nguồn gửi hay tính thời gian thực của dữ liệu, các ECU có thể coi các gói phát lại này là hợp lệ.

Kẻ tấn công cần thực hiện các bước sau:

- Kết nối vào mạng CAN bằng cách can thiệp vật lý vào đường bus.
- Đọc và ghi lại các khung dữ liệu (CAN frames) quan trọng.
- Phát lại các khung dữ liệu này vào thời điểm mong muốn để gây ảnh hưởng đến hành vi của phương tiện.

Hậu Quả

Các cuộc tấn công phát lại có thể gây ra nhiều hậu quả nghiêm trọng như:

- Mở khóa trái phép phương tiện.
- Kích hoạt hoặc vô hiệu hóa hệ thống phanh, túi khí, hoặc các chức năng quan trọng khác.
- Gây nhiễu hoặc làm gián đoạn hoạt động của các ECU khác.

3. Giai đoạn bảo vệ (CAN có phòng thủ)

Để nâng cao khả năng bảo mật cho hệ thống giao tiếp CAN, một cơ chế kiểm tra tính “tươi mới” (freshness) của dữ liệu được tích hợp thông qua việc sử dụng **Bộ đếm thông điệp đơn giản (Simple Message Counter)**. Cơ chế này nhằm ngăn chặn hiệu quả các cuộc tấn công phát lại (Replay Attack) – trong đó kẻ tấn công gửi lại các thông điệp hợp lệ đã được truyền trước đó nhằm đánh lừa hệ thống.

Nguyên lý hoạt động:

Mỗi thông điệp CAN được gắn thêm một trường bộ đếm (counter) có giá trị tăng dần sau mỗi lần truyền. Thiết bị nhận sử dụng giá trị này để xác định tính mới mẻ của thông điệp. Nếu thông điệp có bộ đếm nhỏ hơn hoặc bằng giá trị đã nhận trước đó, thông điệp đó bị từ chối do được xem là lỗi thời hoặc bị phát lại trái phép.

Cách triển khai cụ thể:

- **Gắn trường bộ đếm:** Mỗi thông điệp CAN M_k sẽ được mở rộng để bao gồm một trường bộ đếm nhỏ (thường là 8-bit hoặc 16-bit), được gọi là C_k . Tổng dữ liệu vẫn nằm trong giới hạn 8 byte của trường **Data**.
- **Tăng bộ đếm:** Mỗi khi node phát hợp lệ gửi một thông điệp mới, bộ đếm được tăng lên một đơn vị. Ví dụ: $C_{k+1} = C_k + 1$. Nếu bộ đếm đạt giá trị tối đa, nó sẽ quay vòng về 0 (modulo).
- **Lưu bộ đếm gần nhất:** Thiết bị nhận lưu trữ bộ đếm cao nhất đã nhận được cho mỗi phiên giao tiếp hoặc ID CAN cụ thể, gọi là C_{max} .

- **Kiểm tra bộ đếm:** Khi nhận một thông điệp mới, hệ thống thực hiện kiểm tra:

Nếu $C_{new} \leq C_{max} \Rightarrow$ Từ chối thông điệp (Replay hoặc lỗi thời)

Nếu $C_{new} > C_{max} \Rightarrow$ Chấp nhận thông điệp, cập nhật C_{max}

- **Xử lý vòng lặp:** Dùng byte cuối cùng trong frame data làm biến đếm.

Lợi ích của phương pháp:

- Dễ triển khai trên các hệ thống vi điều khiển nhúng hạn chế tài nguyên.
- Không yêu cầu mã hóa hay xác thực phức tạp, vẫn đảm bảo hiệu quả trong phát hiện phát lại.
- Hoàn toàn tương thích với kiến trúc của giao thức CAN chuẩn (trường dữ liệu không thay đổi cấu trúc khung).

Hạn chế:

- Không phát hiện được các tấn công giả mạo thông điệp mới với bộ đếm hợp lệ.
- Không mã hóa dữ liệu, dễ bị phân tích nếu truyền tin nhạy cảm.
- Cần cơ chế đồng bộ bộ đếm giữa các node trong trường hợp mất kết nối hoặc reset hệ thống.

Kết luận: Việc tích hợp *Simple Message Counter* là một bước quan trọng trong việc nâng cao tính an toàn cho mạng CAN, đặc biệt hữu ích trong các ứng dụng cần bảo vệ tối thiểu với chi phí thấp. Mặc dù không thay thế hoàn toàn các kỹ thuật bảo mật hiện đại như mã hóa hoặc xác thực số, nhưng cơ chế này đóng vai trò như một lớp phòng thủ đầu tiên hiệu quả, dễ triển khai và phù hợp với thiết bị nhúng hạn chế tài nguyên.

4. Tổng kết nguyên lý

Mô hình hoạt động của hệ thống tuân thủ nguyên tắc phát hiện sớm và phản ứng nhanh khi có hành vi bất thường. Các thuật toán được tích hợp nhẹ, phù hợp với vi điều khiển nhúng, nhưng vẫn đảm bảo độ chính xác trong việc nhận diện các kiểu tấn công phổ biến trong mạng CAN. Sự so sánh giữa ba giai đoạn vận hành (chưa có bảo vệ, bị tấn công và có bảo vệ) giúp đánh giá rõ ràng hiệu quả của các kỹ thuật phòng thủ được đề xuất.

V. Mã nguồn phát triển của hệ thống

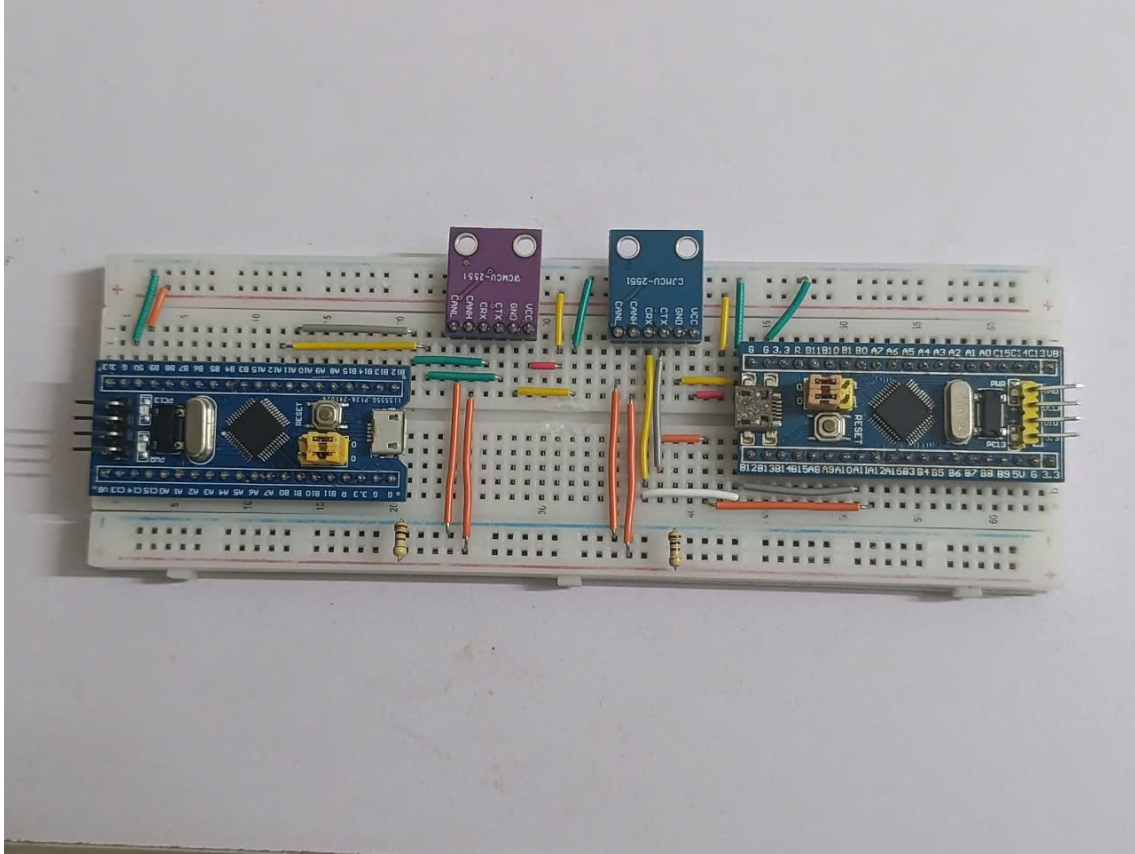
Toàn bộ mã nguồn, bao gồm chương trình điều khiển vi điều khiển, giao diện website mô phỏng tấn công và phòng thủ, cũng như tài liệu hướng dẫn, đều được công khai tại GitHub.

- **Kho lưu trữ GitHub:** github.com/quachtiensinh27/CAN_CYBER_SECURITY
- **Nội dung chính bao gồm:**
 - Mã lập trình cho hệ thống truyền nhận CAN không có bảo vệ và có bảo vệ, viết bằng thanh ghi cho STM32F103C8T6.
 - Giao diện web cho mô phỏng tấn công và phòng thủ.
 - Video minh họa, tài liệu README và hướng dẫn cấu hình.
- **Mục tiêu:** Giúp người học, giảng viên, và các nhà nghiên cứu có thể dễ dàng tái tạo, thử nghiệm, hoặc cải tiến hệ thống trên các nền tảng phần cứng tương tự.

Chương 6: Thực thi, đo lường và đánh giá

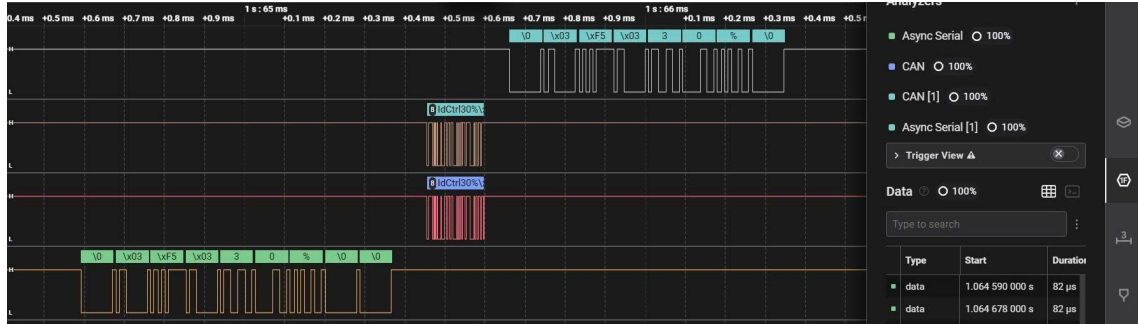
I. Giao tiếp 2 chiều

Trong giao tiếp 2 chiều, chúng tôi thực hiện truyền các gói tin qua lại giữa 2 vi điều khiển trong mạng CAN.



Hình 19: Mạch thực hành giao tiếp giữa 2 vi điều khiển.

Tín hiệu mô phỏng truyền đi có $CAN_id = 0x03F5$ và data là '30%'. Sử dụng logic analyzer để đọc tín hiệu UART cũng như tín hiệu CAN trong quá trình truyền, kết quả thu được như hình dưới đây:



Hình 20: Giao tiếp giữa 2 vi điều khiển.

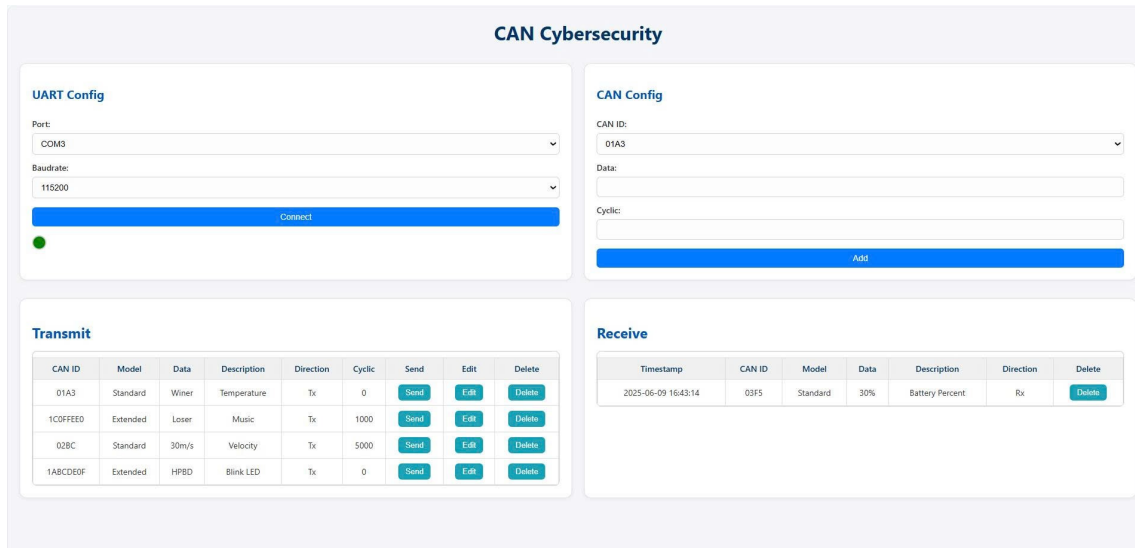
Quan sát tín hiệu ở hình trên, ta thấy ban đầu sẽ có tín hiệu UART truyền từ PC xuống vi điều khiển (tín hiệu dưới cùng). Cấu trúc frame của UART giống với cấu trúc đã được cấu hình, gồm:

- 1 byte đầu: xác định Model để xác định các byte tiếp theo cho CAN_id. Giá trị 0x00 -> là dạng Standard -> đọc 2 byte tiếp theo cho CAN_id.
- 2 bytes tiếp: CAN_id. CAN_id của tín hiệu có giá trị là 0x03F5, giống với giá trị được cấu hình để gửi đi trên PC.
- 1 byte tiếp dùng để xác định data_length, có giá trị 0x03 -> xác định 3 bytes tiếp theo là data.
- 3 bytes data: giá trị hiển thị các byte với mã ascii là '30%' -> giống với setup trên PC.
- Cuối cùng là 2 bytes Cyclic, dùng để xác định việc gửi tuần hoàn (ms) hay 1 lần. Giá trị trên ảnh là 0x0000 -> tín hiệu được gửi 1 lần duy nhất.

Tiếp đến là các tín hiệu CAN đọc được ở 2 đầu vi điều khiển, các tín hiệu đọc được đều có data và id giống với cấu hình, thể hiện việc truyền quan CAN bus đã đúng tín hiệu.

Cuối cùng là tín hiệu UART của bên nhận gửi lên PC, các trường vẫn giữ nguyên, chỉ có trường 2 bytes Cyclic cuối cùng được thay bằng 1 byte cảnh báo tấn công. Giá trị byte là 0x00 tức là đang không bị tấn công.

Tín hiệu nhận được thể hiện trên web nhận đúng với kết quả gửi đi:



Hình 21: Tín hiệu nhận được trên Web.

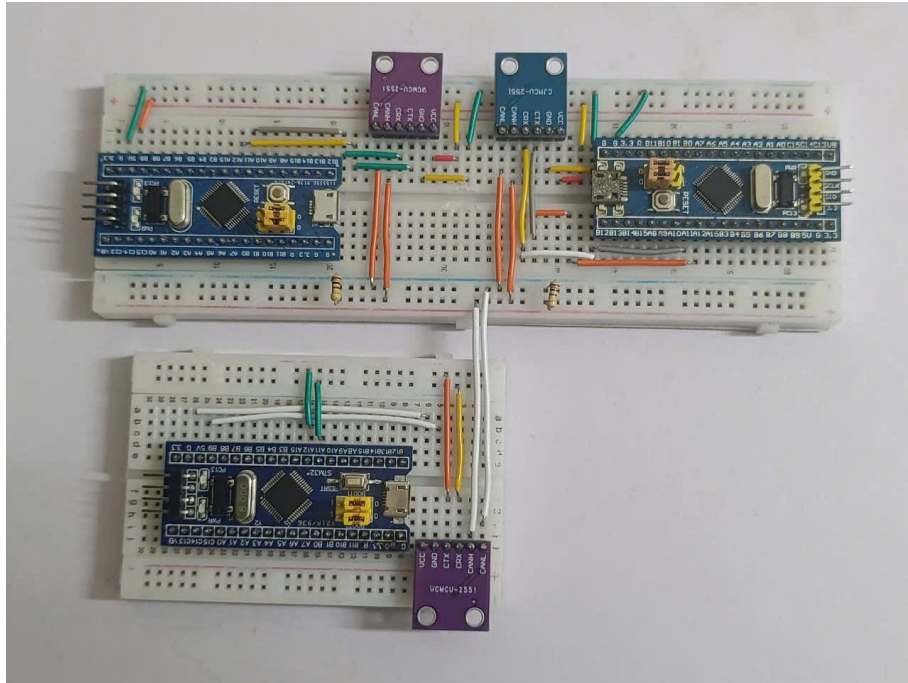
Video mô phỏng truyền nhận qua mạng CAN:

[Video truyền 2 chiều trong CAN](#)

Nhận xét: Hệ thống đã thành công trong việc truyền và nhận bằng giao thức Can giữa 2 vi điều khiển. Tín hiệu truyền tốt, tốc độ gần sát với thực tế: Baurate Uart: 115200, tốc độ Can : 500000.

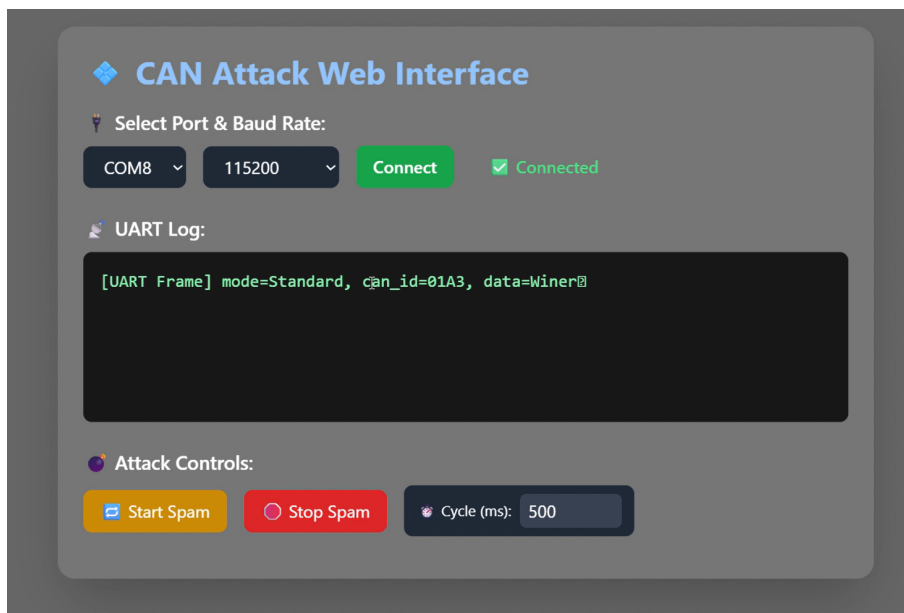
II. Tấn công

Để mô phỏng tấn công mạng CAN, một node thứ 3 được thêm vào để thực hiện chức năng của hacker như hình dưới đây:



Hình 22: Mạch giao tiếp CAN thêm phần tử Hacker.

Sau khi có tín hiệu giao tiếp trong mạch CAN, hacker sẽ đọc tín hiệu và lưu lại tín hiệu mới nhất. Tín hiệu sẽ được hiển thị trên giao diện ở dưới:

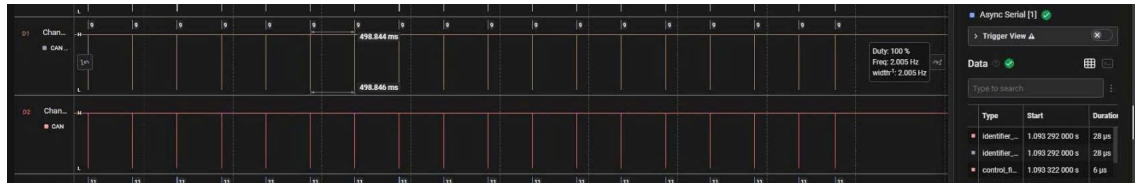


Hình 23: Giao diện tấn công với cyclic 500ms.

Hacker lưu lại thông tin về CAN_id và data lại. Tín hiệu được gửi đi có data là 'Winer', tuy nhiên tín hiệu mà hacker đọc được lại là 'winer[?]'. Ký tự cuối cùng chính là byte counter trong frame CAN dùng để phòng chống hacker. Trong mạch truyền nhận thông thường sẽ có thể tách biến counter này ra trong các byte data, tuy nhiên

hacker không thể biết cách phân biệt nên vẫn lưu lại giá trị byte này như 1 data. Đó là lý do có ký tự ở sau 'winer'.

Khi ấn Start Spam thì hacker sẽ tiến hành tấn công với cyclic tương ứng trên giao diện. Ở đây, hacker sẽ replay attack với chu kỳ 500ms liên tục cho đến khi ấn Stop. Tín hiệu trên CAN bus tương ứng cũng sẽ hiển thị liên tục các tín hiệu tấn công của hacker:



Hình 24: Tín hiệu tấn công trên CAN bus với chu kỳ 500ms.

Dưới đây là video thể hiện quá trình bị tấn công của bên nhận trong mạng CAN.

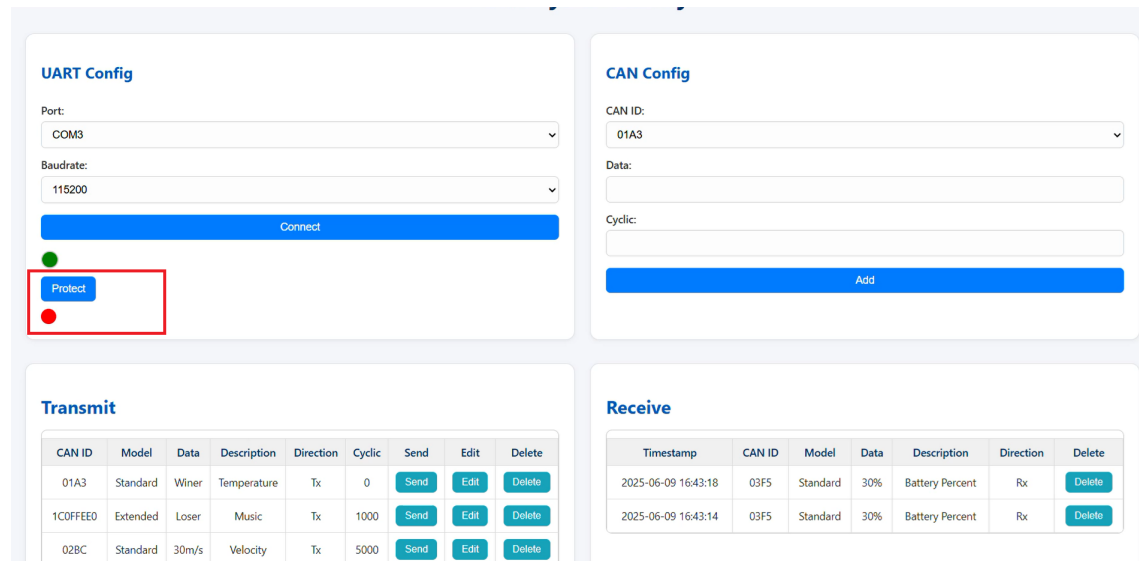
Video mô phỏng quá trình bị tấn công trong CAN:

[Video truyền CAN và bị Replay Attack](#)

Nhận xét: Hệ thống đã bị tấn công: Vì User1 chỉ gửi 1 tín hiệu cho User2 nhưng bị hacker nhận được và spam lại cho User2 khiến User2 nhận tín hiệu lặp lại liên tục mà thực tế User1 không truyền. Đây chính là bị tấn công "Replay attack"

III. Phòng thủ

Với hệ thống mạng CAN có phòng thủ, mạch phần cứng vẫn tương tự như phần trên. Tuy nhiên giao diện phần mềm được cập nhật thêm cơ chế phòng thủ (Protect) như hình dưới đây:



Hình 25: Giao diện với cơ chế phòng thủ.

Hệ thống có thêm 1 cơ chế phát hiện và cảnh báo khi nhận được tín hiệu tấn công. Khi bật chế độ phòng thủ lên thì hệ thống vẫn sẽ hiện cảnh báo, tuy nhiên sẽ không nhận các tín hiệu được phát hiện là tấn công.

Dưới đây là mô phỏng quá trình bảo mật trong mạng CAN.

Video mô phỏng cơ chế phòng thủ:

[Video phòng thủ](#)

Nhận xét: Hệ thống đã thành công trong việc phòng thủ khỏi "Replay attack" : Khi bị Hacker spam tín hiệu không hợp lệ, User2 với hệ thống phòng thủ đã ngăn cản và chỉ nhận các tín hiệu đó và vẫn nhận được các tín hiệu hợp lệ của User1.

Chương 7: Tổng kết và hướng phát triển

I. Tổng kết

Bài tập lớn "CyberCAN Defense – Phát hiện và phòng thủ xâm nhập mạng nội bộ ô tô" tập trung nghiên cứu và xây dựng một hệ thống giám sát, phát hiện và ngăn chặn tấn công trên giao thức truyền thông CAN – vốn là xương sống trong các hệ thống điều khiển điện tử của xe ô tô hiện đại.

Trong quá trình thực hiện, nhóm đã đạt được các kết quả nổi bật sau:

- **Nghiên cứu lý thuyết nền tảng:** Nhóm đã nắm vững kiến thức về giao thức CAN, bao gồm cấu trúc khung dữ liệu, phương thức arbitration, truyền thông theo cơ chế broadcast, xử lý lỗi, mức điện áp vật lý và khả năng mở rộng mạng.
- **Xây dựng mô hình hệ thống CAN thực tế:** Sử dụng vi điều khiển phổ biến như STM32F103C8T6, kết hợp với transceiver MCP2551, nhóm đã thiết lập mạng CAN 2 chiều hoàn chỉnh giữa hai thiết bị. Các chương trình điều khiển đều được viết ở mức thanh ghi để hiểu sâu hơn hoạt động nội tại của phần cứng.
- **Thiết kế hệ thống truyền nhận cơ bản và có bảo vệ:**
 - Giao diện truyền nhận CAN chưa có bảo vệ được xây dựng nhằm mô phỏng hoạt động bình thường giữa các thiết bị trong mạng.
 - Giao diện có bảo vệ giúp ngăn chặn các gói tin sai lệch từ bên thứ 3 - hacker.
- **Mô phỏng tấn công và phòng thủ:**
 - Xây dựng mô hình tấn công giả lập theo kiểu *Replay Attack*, có thể can thiệp vào mạng bằng cách gửi các gói tin đã ghi lại một cách liên tục.
 - Triển khai cơ chế phòng thủ để phát hiện những gói tin bất thường, từ đó đưa ra cảnh báo và ngăn chặn lây lan.
- **Xây dựng giao diện trực quan bằng website:**
 - Các website tương tác mô phỏng quá trình truyền nhận, tấn công và phòng thủ trên giao thức CAN, giúp việc trình bày và minh họa kết quả trở nên sinh động và dễ hiểu hơn.
- **Minh họa kết quả qua video thực nghiệm:** Ghi lại toàn bộ quá trình truyền CAN cơ bản, bị tấn công và phòng thủ thành công nhằm chứng minh tính khả thi của hệ thống.

Kết quả từ dự án cho thấy hệ thống có thể phát hiện và loại bỏ hiệu quả các gói tin không hợp lệ, nâng cao độ tin cậy của mạng truyền thông trong xe – một bước tiến quan trọng trong việc bảo vệ hệ thống an toàn điều khiển ô tô trước các rủi ro an ninh mạng ngày càng tăng cao.

II. Hướng phát triển trong tương lai

Để mở rộng tiềm năng và ứng dụng thực tiễn của dự án, nhóm đề xuất một số hướng phát triển như sau:

- **Tăng cường bảo mật với mã hóa:** Triển khai các thuật toán mã hóa đối xứng nhẹ (Lightweight Cryptography) phù hợp với vi điều khiển công suất thấp để mã hóa nội dung gói tin CAN.
- **Tích hợp hệ thống phát hiện bất thường dựa trên Machine Learning:**
 - Huấn luyện mô hình học máy (như SVM, Random Forest, hoặc mạng neural đơn giản) để phát hiện mẫu dữ liệu bất thường trong quá trình hoạt động mạng.
 - Có thể áp dụng học không giám sát (unsupervised learning) để phát hiện tấn công chưa từng thấy (zero-day).
- **Phát triển cơ chế xác thực mạnh hơn:**
 - Tích hợp xác thực dựa trên ID động, thẻ bảo mật phần cứng (TPM), hoặc kết hợp nhiều yếu tố để chống giả mạo node.
- **Xây dựng hệ thống giám sát thời gian thực:**
 - Phát triển dashboard theo thời gian thực để hiển thị trạng thái mạng, nhật ký các sự kiện bất thường, và đưa ra phản hồi tự động (như cô lập node nghi ngờ).
- **Áp dụng trên mô hình ô tô thật hoặc mô hình giả lập thương mại:**
 - Thử nghiệm hệ thống trên mô hình ô tô như car simulator, hoặc các nền tảng phần mềm giả lập (Vector CANoe, Simulink Vehicle Network Toolbox) để đánh giá khả năng hoạt động trong môi trường gần thực tế.
- **Hướng đến hệ thống đa giao thức:** Ngoài CAN, các giao thức khác trong xe như LIN (cho mạng cục bộ tốc độ thấp), FlexRay (truyền thời gian thực) và Automotive Ethernet cũng cần được nghiên cứu để xây dựng hệ thống bảo mật toàn diện hơn.

Với sự phát triển nhanh chóng của ô tô điện, xe tự hành và kết nối Internet vạn vật (IoT), bảo mật mạng nội bộ trong xe là yếu tố không thể thiếu để đảm bảo an toàn vận hành. Dự án CyberCAN Defense chính là bước khởi đầu để nhóm có thể chuẩn bị cho những dự án liên quan tiếp theo.

Tài liệu tham khảo

1. STM32F103 Reference manual - STMicroelectronics.
2. MCP2551 datasheet - Microchip.
3. Introduction to the Controller Area Network (CAN) - Texas Instruments.
4. Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., Savage, S., ... & Kohno, T. (2011). Comprehensive Experimental Analyses of Automotive Attack Surfaces. USENIX Security Symposium.
5. Wolf, D., & Gendrullis, T. (2012). Design, Implementation, and Evaluation of a Vehicular Hardware Security Module. International Conference on Information Security and Cryptology.
6. Groza, B., Murvay, P. S., & Van Herrewege, A. (2020). LiDAR to CAN Bus: Attack Vectors and Countermeasures in Modern Vehicles. IEEE Communications Surveys & Tutorials.

ĐÓNG GÓP

Tên	Mã sinh viên	Mô tả công việc	Tỷ lệ đóng góp
Quách Ngọc Quang	22022132	Thiết kế giao diện web gửi UART từ PC đến vi điều khiển và xử lý truyền dữ liệu từ vi điều khiển lên bus CAN	25%
Nguyễn Tuấn Phong	22022160	Thiết kế web nhận UART từ vi điều khiển về PC và xử lý tín hiệu CAN nhận được tại vi điều khiển	25%
Nguyễn Minh Phúc	22022178	Thiết kế và triển khai các cơ chế bảo mật, phòng thủ cho hệ thống	25%
Đoàn Đức Mạnh	22022167	Thiết kế và triển khai phương pháp tấn công từ bên thứ ba nhằm kiểm thử bảo mật	25%